

Model Capacity, LASSO and Cross Validation

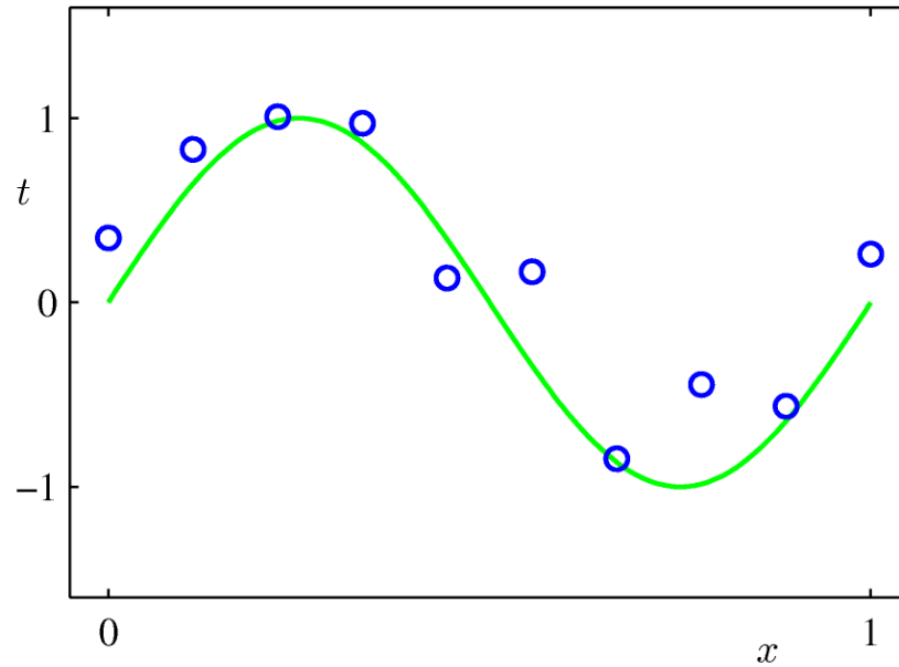
Capacity of a model

Model capacity is ability to fit variety of functions

- Model with Low capacity struggles to fit training set
- A High capacity model can overfit by memorizing properties of training set not useful on test set
 - When model has higher capacity, it overfits
- One way to control capacity of a learning algorithm is by choosing the hypothesis space
 - set of functions that the learning algorithm is allowed to select as being the solution
 - E.g., the linear regression algorithm has the set of all linear functions of its input as the hypothesis space
 - We can generalize to include polynomials in its hypothesis space which increases model capacity

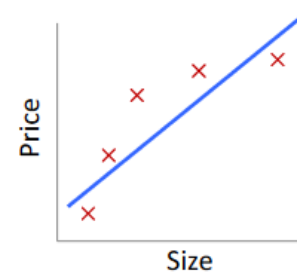
Capacity of Polynomial Curve Fits

$$y = \theta_0 x^0 + \theta_1 x + \theta_2 x^2 + \cdots + \theta_p x^p = \sum_{j=0}^p \theta_j x^j$$

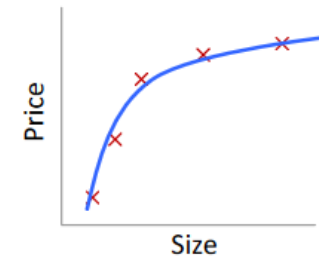


Appropriate Capacity

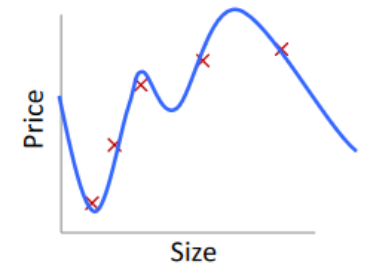
- Machine Learning algorithms will perform well when their capacity is appropriate for the true complexity of the task that they need to perform and the amount of training data they are provided with
- Models with insufficient capacity are unable to solve complex tasks
- Models with high capacity can solve complex tasks, but when their capacity is higher than needed to solve the present task, they may overfit



$\theta_0 + \theta_1 x$
Underfitting
(high bias)



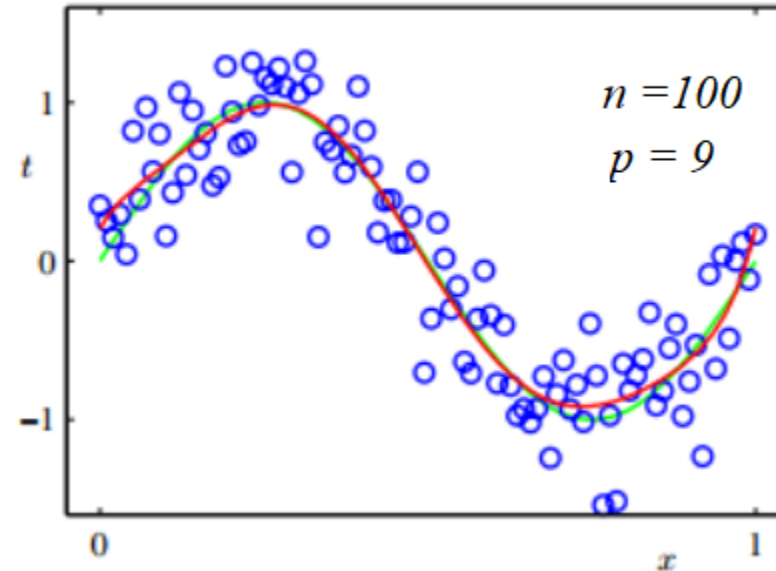
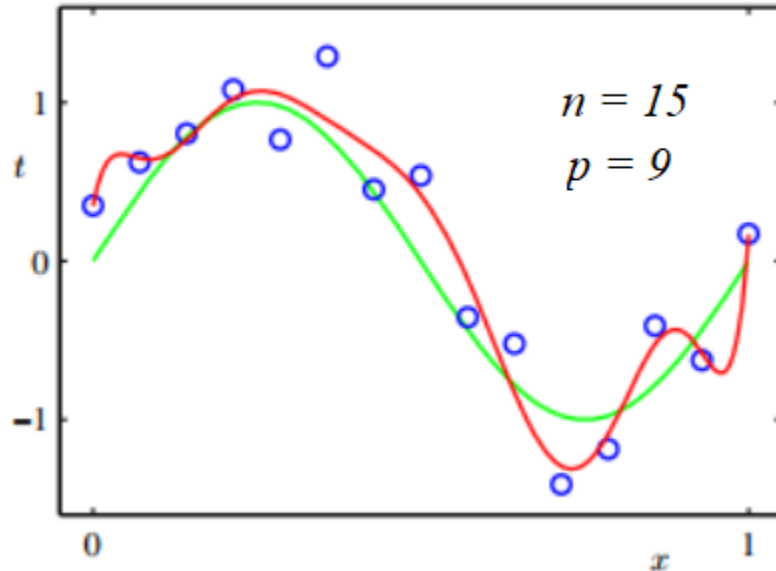
$\theta_0 + \theta_1 x + \theta_2 x^2$
Correct fit



$\theta_0 + \theta_1 x + \theta_2 x^2 + \theta_3 x^3 + \theta_4 x^4$
Overfitting
(high variance)

Reducing Overfitting Issue

- Reduce the complexity or model capacity
- Regularization
- Increase the amount of data (5 or 10 times the number of parameters in the model)

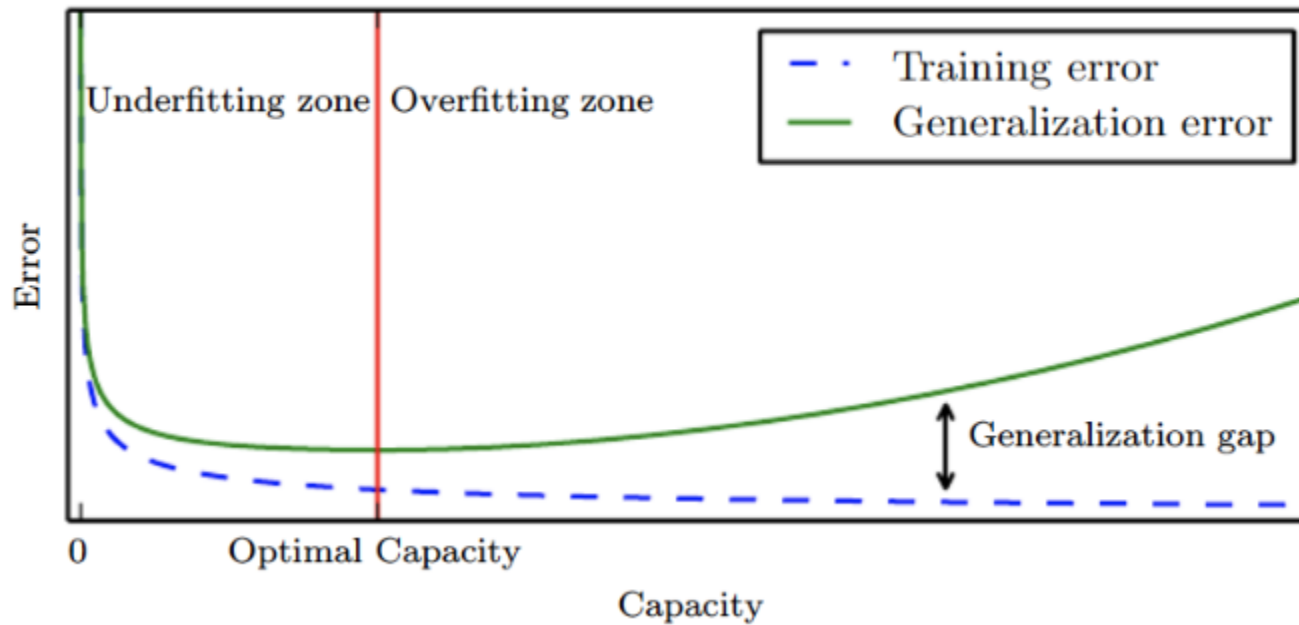


The Effective Number of Parameters

- Stack the outcomes $\mathbf{y} = [y^{(1)}, \dots, y^{(n)}]^T$ and predictions $\hat{\mathbf{y}}$.
- For a linear fitting method $\hat{\mathbf{y}} = \mathbf{S}\mathbf{y}$
- $\mathbf{S}$ is an $n \times n$ matrix depends on the input vector $x^{(i)}$
- The number of effective parameters is defined as $d(\mathbf{S}) = \text{trace}(\mathbf{S})$

Typical Generalization Error

- Relationship between capacity and error
- Typically generalization error has a U-shaped curve



Least Absolute Shrinkage and Selection Operator (LASSO)

$$\hat{\theta} = \underset{\theta}{\operatorname{argmin}} \quad \|\vec{y} - \vec{X}\vec{\theta}\|_2^2 + \lambda \|\vec{\theta}\|_1$$

Special case: $\vec{X}^T \vec{X} = \mathbf{I}$.

$$J(\theta) = \vec{y}^T \vec{y} + \vec{\theta}^T \vec{X}^T \vec{X} \vec{\theta} - 2 \vec{y}^T \vec{X} \vec{\theta} + \lambda \|\vec{\theta}\|_1,$$

$$= \vec{y}^T \vec{y} + \vec{\theta}^T \vec{\theta} - 2 \vec{y}^T \vec{X} \vec{\theta} + \lambda \|\vec{\theta}\|_1,$$

$$J(\theta) = \vec{y}^T \vec{y} + \sum_{j=1}^d (\theta_j^2 - 2 \vec{y}^T \vec{x}_j \theta_j + \lambda |\theta_j|)$$

$\vec{x}_j = j\text{th column of } \vec{X}$

$$J(\theta) = y^T y + \sum_{j=1}^d (\theta_j^2 - 2 y^T x_j \theta_j + \lambda |\theta_j|)$$

For $\theta_j > 0$:- $2\theta_j - 2y^T x_j + \lambda = 0$

$$\hat{\theta}_j = y^T x_j - \frac{\lambda}{2}$$

For $\theta_j < 0$:- $2\theta_j - 2y^T x_j - \lambda = 0$

$$\hat{\theta}_j = \underline{y^T x_j + \frac{\lambda}{2}}$$

For $\theta_j = 0 \rightarrow \hat{\theta}_j = 0$

Ordinary Least Square soln $\hat{\theta}_j^0 = y^T x_j$

$$\hat{\theta}_j = \begin{cases} \hat{\theta}_j^0 + \frac{\lambda}{2} \\ \hat{\theta}_j^0 - \frac{\lambda}{2} \\ 0 \end{cases}$$

if $\hat{\theta}_j < 0$

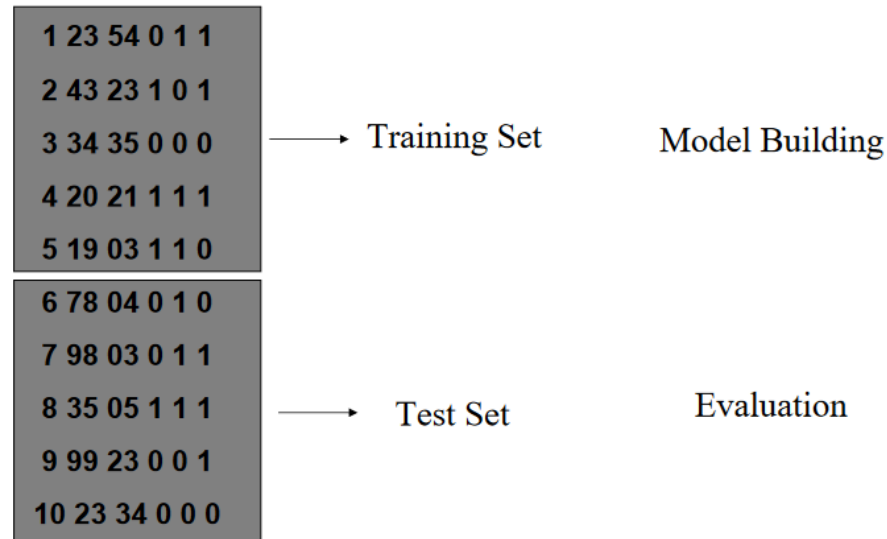
if $\hat{\theta}_j > 0$

else

Soft-Thresholding

Training and Testing Sets

- Training set is used to build the model
- Test set left aside for evaluation purposes
- Ideal: different data set to test if model generalizes to other settings
- If data are abundant, then there is no need to “recycle” cases



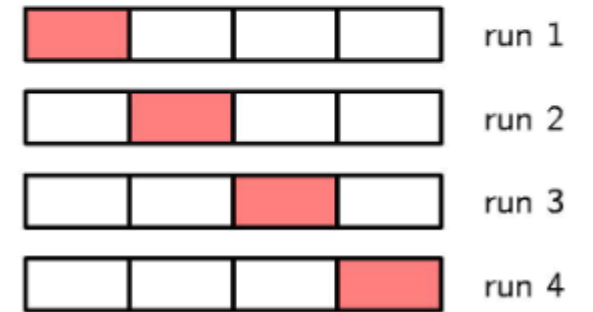
Cross-Validation

- The simplest and most widely used method for estimating prediction error is cross-validation.
- When data set is too small, dividing into a fixed training set and fixed testing set is problematic
 - If it results in a small test set
 - Small test set implies statistical uncertainty around the estimated average test error
 - Cannot claim algorithm A works better than algorithm B for a given task
- k - fold cross-validation
 - Partition the data into k non-overlapping subsets
 - On trial i , i th subset of data is used as the test set
 - Rest of the data is used as the training set

k - fold Cross-Validation

- Supply of data is limited
- All available data is partitioned into k groups (folds)
- $k - 1$ groups are used to train and evaluated on remaining group
- Repeat for all k choices of held-out group
- Performance scores from k runs are averaged

$k=4$



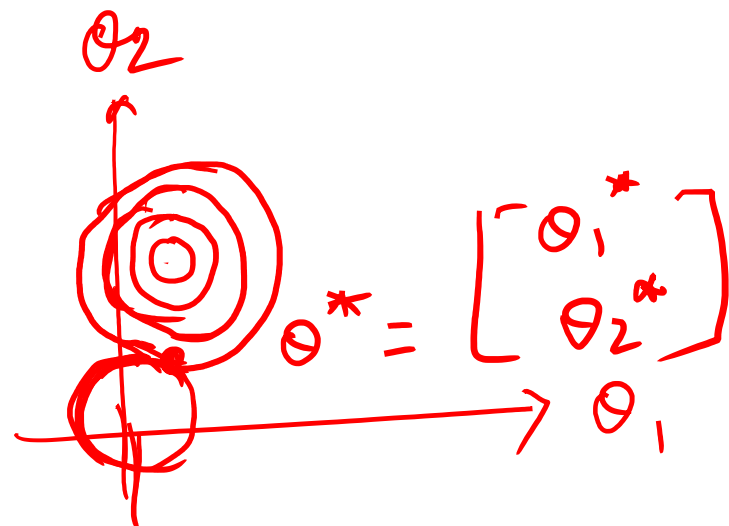
k - fold Cross-Validation

- $\kappa: \{1, 2, \dots, n\} \mapsto \{1, 2, \dots, k\}$ -- An indexing function that indicates the partition to which observation i is allocated by the randomization.
- The fitted function is $\hat{f}^{-k}(x)$ be the fitted function, computed with the k th part of the data removed.
- The cross-validation estimate of the prediction error is

$$CV = \frac{1}{n} \sum_{i=1}^n L(y_i, \hat{f}^{-\kappa(i)}(x_i))$$

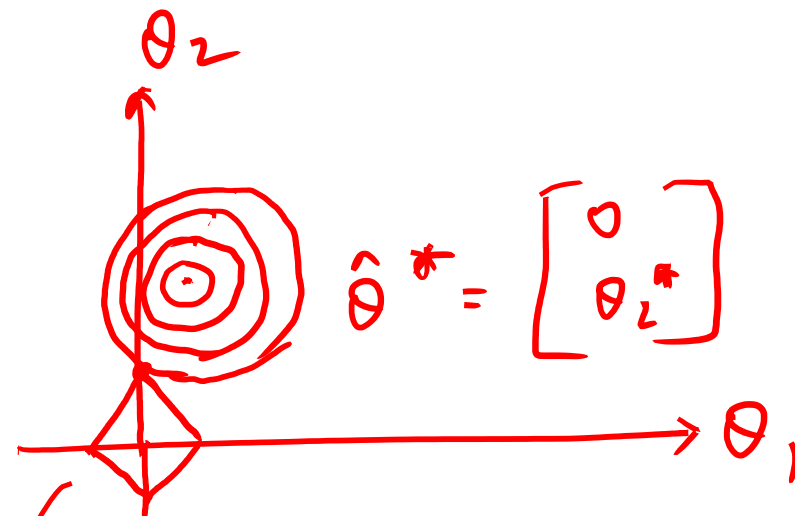
- Typical choices of K are 5 or 10.
- $k = n$ is known as *leave-one-out* cross-validation.

LASSO & Cross Validation



Ridge Regression

$$\|\theta\|_2^2$$



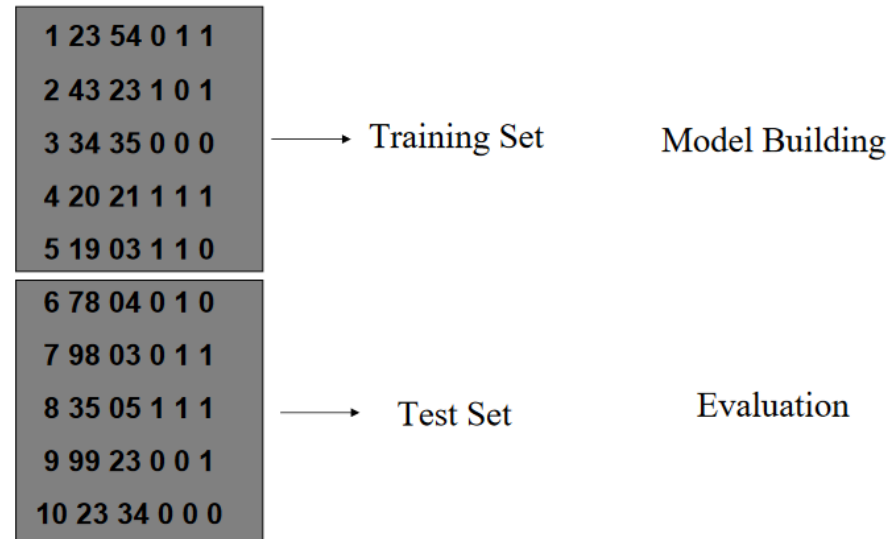
LASSO

$$\|\theta\|_1$$

Sparsity

Training and Testing Sets

- Training set is used to build the model
- Test set left aside for evaluation purposes
- Ideal: different data set to test if model generalizes to other settings
- If data are abundant, then there is no need to “recycle” cases



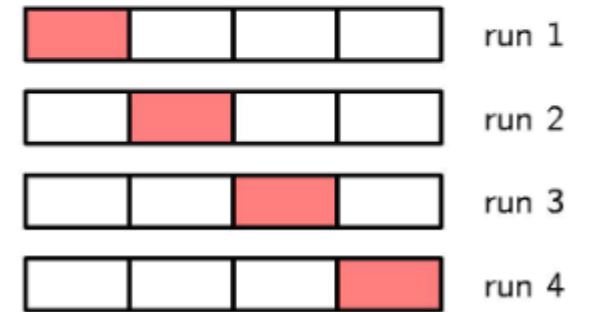
Cross-Validation

- The simplest and most widely used method for estimating prediction error is cross-validation.
- When data set is too small, dividing into a fixed training set and fixed testing set is problematic
 - If it results in a small test set
 - Small test set implies statistical uncertainty around the estimated average test error
 - Cannot claim algorithm A works better than algorithm B for a given task
- k - fold cross-validation
 - Partition the data into k non-overlapping subsets
 - On trial i , i th subset of data is used as the test set
 - Rest of the data is used as the training set

k - fold Cross-Validation

- Supply of data is limited
- All available data is partitioned into k groups (folds)
- $k - 1$ groups are used to train and evaluated on remaining group
- Repeat for all k choices of held-out group
- Performance scores from k runs are averaged

$k=4$



k - fold Cross-Validation

- $\kappa: \{1, 2, \dots, n\} \mapsto \{1, 2, \dots, k\}$ -- An indexing function that indicates the partition to which observation i is allocated by the randomization.
- The fitted function is $\hat{f}^{-k}(x)$ be the fitted function, computed with the k th part of the data removed.
- The cross-validation estimate of the prediction error is

$$CV = \frac{1}{n} \sum_{i=1}^n L(y_i, \hat{f}^{-\kappa(i)}(x_i))$$

- Typical choices of K are 5 or 10.
- $k = n$ is known as *leave-one-out* cross-validation. (LOOCV)

$$\hat{\theta}_1, \hat{\theta}_2, \dots$$

$$\boxed{E[\hat{\theta}] = \theta} \leftarrow \text{original parameter.}$$

↓
Estimator $\rightarrow$ unbiased.

$$E[\hat{\theta}] - \theta \neq 0 \rightarrow \text{Biased.}$$

$$E[\hat{\theta}] - \theta = \text{Bias.}$$

LOOCV :-

$$x^{(i)} = A + w^{(i)} \quad i=1, \dots, n$$

$A =$ unknown DC level

$$w^{(i)} \sim \mathcal{N}(0, \sigma^2)$$

MLE :-

$$\hat{A} = \frac{1}{n} \sum_{i=1}^n x^{(i)}$$

$$E[\hat{A}] = \frac{1}{n} \sum_{i=1}^n E[x^{(i)}]$$

Unbiased

$$= \frac{1}{n} \sum_{i=1}^n A = \frac{nA}{n} = A$$

$$x^{(i)} = A + w^{(i)}$$
$$E[x^{(i)}] = A$$

$$\text{var}(\hat{A}) = \frac{1}{n^2} \sum_{i=1}^n \text{var}(x^{(i)}) = \frac{\sigma^2}{n}$$

Leave One Out Estimator :-

$$\hat{A}_{(-k^{(i)})} = \frac{1}{n-1} \sum_{i \neq n} x^{(i)}$$

$$E[\hat{A}_{(-k^{(i)})}] = \frac{1}{n-1} \sum_{i \neq n} E[x^{(i)}] = A$$

$$\text{Var}(\hat{A}_{-k(i)}) = \frac{1}{(n-1)^2} \cdot (n-1) \sigma^2 = \frac{\sigma^2}{n-1}$$

$$\text{Var}(\hat{A}_{-k(i)}) \geq \text{Var}(\hat{A})$$

Linear Error:- $e^{(i)} = x^{(i)} - \hat{A}_{-k(i)}$

$$x^{(i)} = A + w^{(i)}$$

$$e^{(i)} = A + w^{(i)} - \frac{1}{n-1} \sum_{n \neq i} [A + w^{(i)}]$$

$$e^{(i)} = w^{(i)} - \frac{1}{n-1} \sum_{n \neq i} w^{(i)}$$

$$E[e^{(i)}] = 0$$

$$\text{Var}(e^{(i)}) = \text{Var}(w^{(i)}) + \text{Var}\left(\frac{1}{n-1} \sum_{n \neq i} w^{(i)}\right)$$

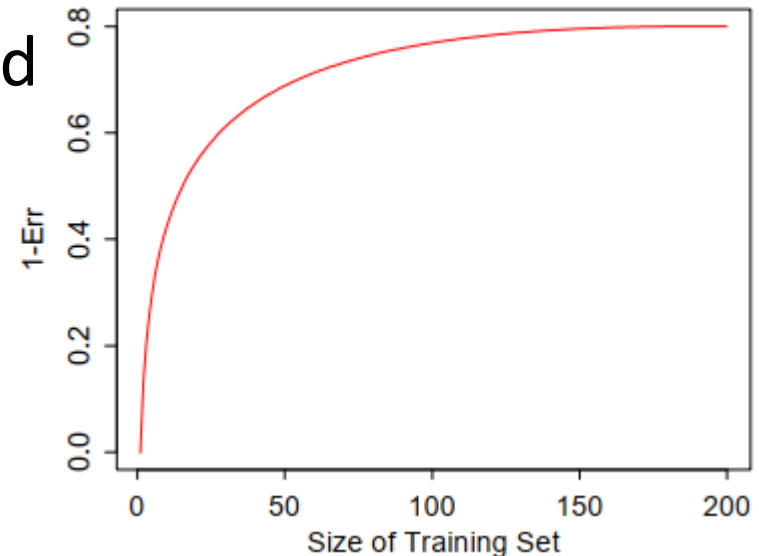
$$= \sigma^2 + \frac{\sigma^2}{n-1} = \frac{n}{n-1} \sigma^2$$

For small n , fluctuations will be noticeable.

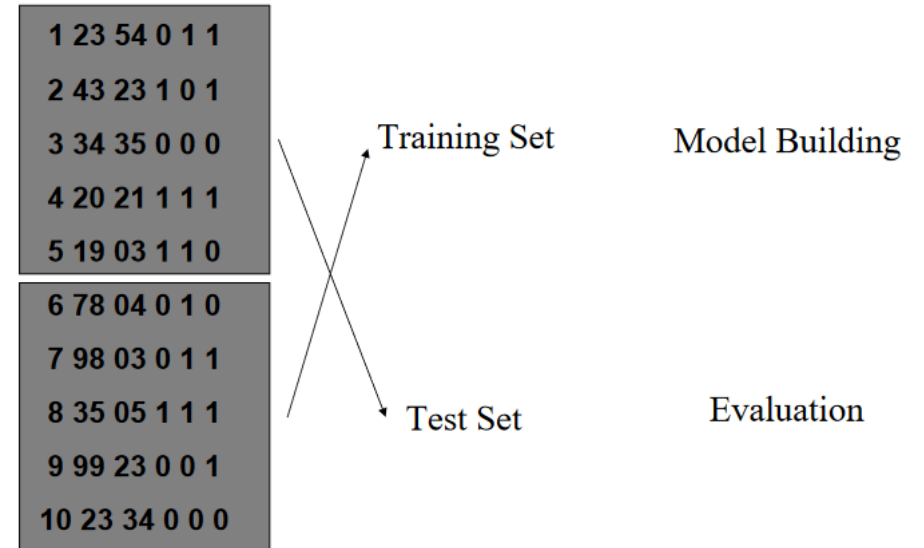
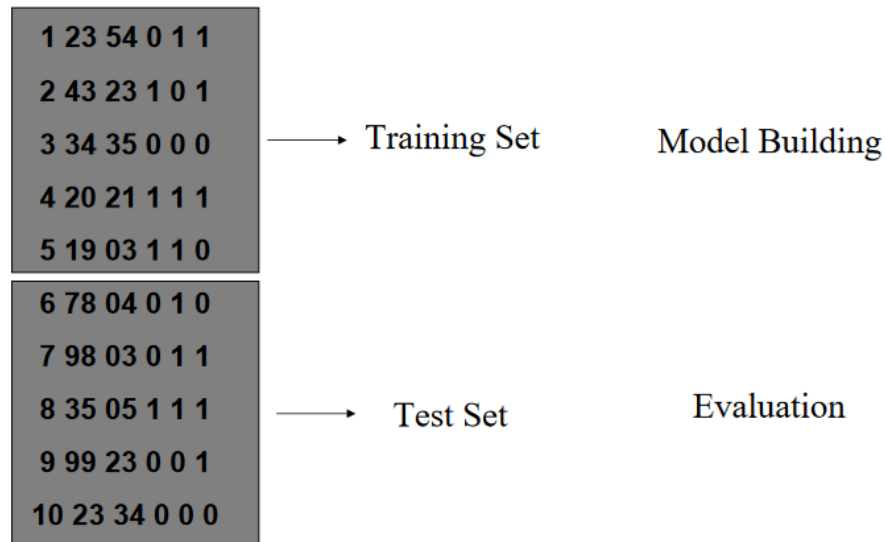
" large n , " not be "

What value should we choose for k ?

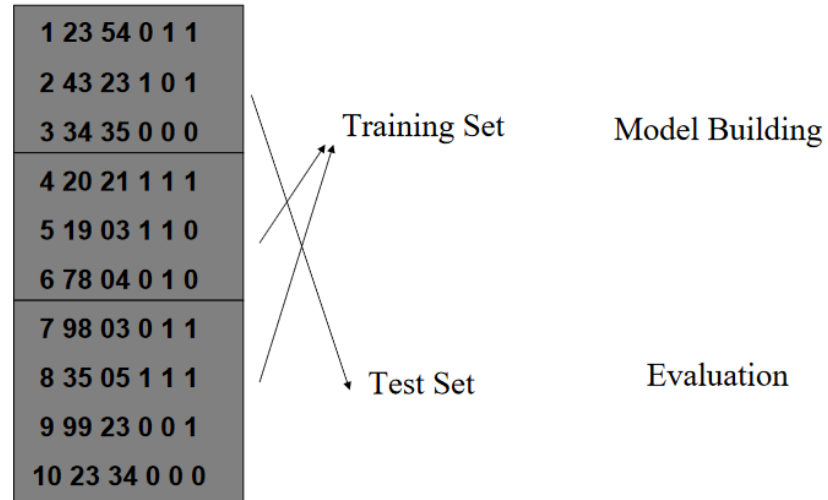
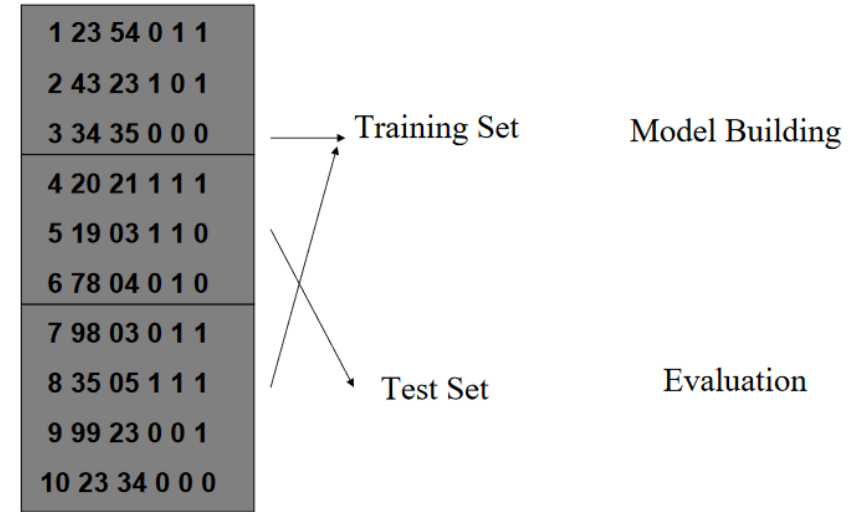
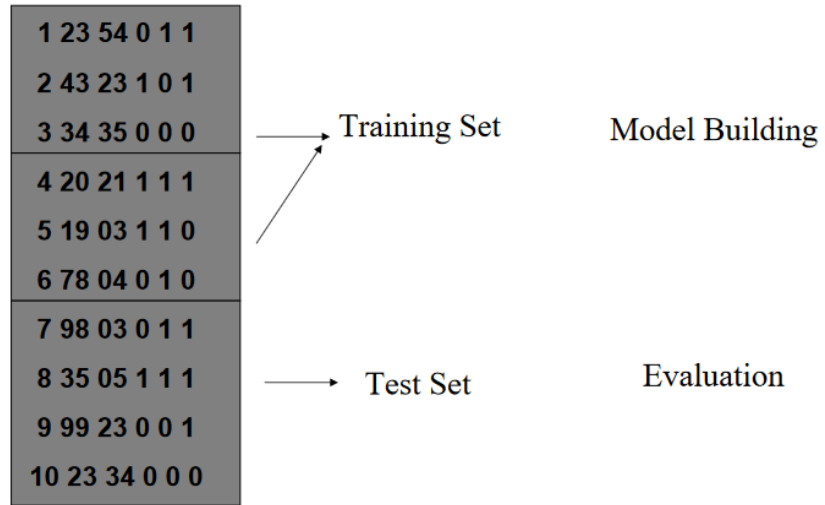
- $k = n$, CV is approximately unbiased for the true prediction error, but can have high variance because all the training sets are so similar to one another
- $k = 5$, CV shall have lower variance, but bias could be problem, depending on how the performance of the learning method varies with the size of the training set.
- For 200 observations, 5-fold cross-validation would not suffer from much bias
- For 50 observations, 5-fold cross-validation would underestimate 1-Err.



$k=2$ Example



$k=3$ Example



Bootstrap

Bootstrap Motivation

- Sometimes it is not possible to collect many samples from a population
- Sometimes it is not correct to assume a certain distribution for the population
- Goal: Assess sampling variation

Bootstrap Methods

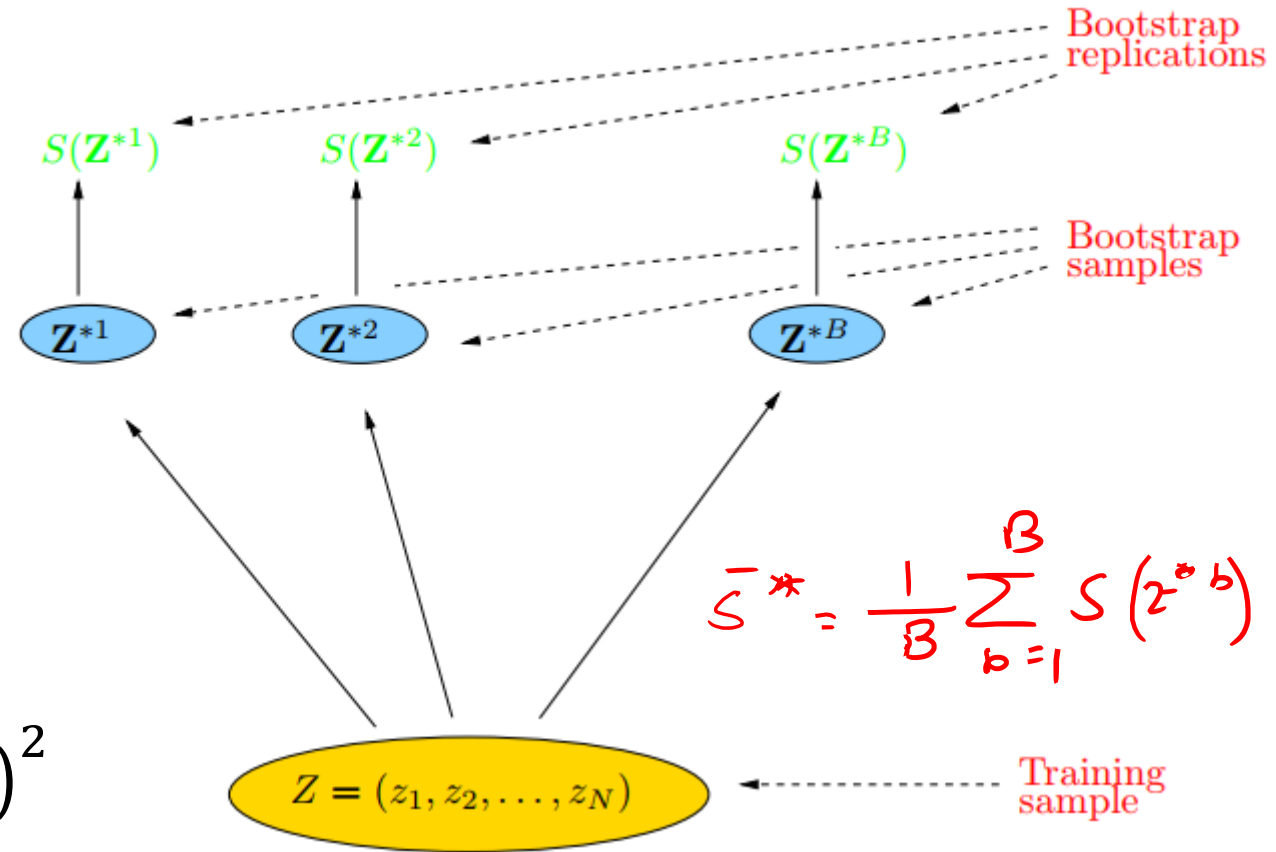
- The bootstrap is a general tool for assessing statistical accuracy.
- Lets denote the training set by $Z = (z_1, z_2, \dots, z_N)$ where $z_i = (x_i, y_i)$
- Randomly draw datasets with replacement from the training data, each sample the same size as the original training set.
- If you do it B times, producing B bootstrap datasets.
- Refit the model to each of the bootstrap datasets, and examine the behaviour of the fits over B replications.

Bootstrap Methods

- $S(Z)$ is prediction at some input Point of Z
- We can estimate any aspect of the distribution

$$\widehat{Var}[S(Z)] = \frac{1}{B-1} \sum_{b=1}^B (S(Z^{*b}) - \bar{S}^*)^2$$

- The prediction error: $\widehat{Err}_{boot} = \frac{1}{B} \frac{1}{N} \sum_{b=1}^B \sum_{i=1}^N \underline{L(y_i, \hat{f}^{*b}(x_i))}$
- $\hat{f}^{*b}(x_i)$ is the predicted value at x_i , from the model fitted to the b th bootstrap dataset.



Bootstrap Example

- Adapted from Fox (1997) “Applied Regression Analysis”
- Goal: Estimate mean difference between Male and Female
- Four pairs of observations are available:

Observ.	Male	Female	Differ. Y
1	24	18	6
2	14	17	-3
3	40	35	5
4	44	41	3

Mean = 2.75

Std Dev = 4.04

Sample	Y_1^*	Y_2^*	Y_3^*	Y_4^*	$\bar{Y}^*$
1	6	6	6	6	6.00
2	6	6	6	-3	3.75
3	6	6	6	5	5.75
..					
100	-3	5	6	3	2.75
101	-3	5	-3	6	1.25
...					
255	-3	3	3	5	3.5
256	3	3	3	3	3.00

Measuring Bias and Variance

- Create B bootstrap variants of the dataset
- Find out the predictions for each of the B bootstrap variants
- Compute the average of all the predictions $\overline{S}^* = \frac{1}{B} \sum_{b=1}^B S(\mathbf{Z}^{*b})$
- Bias: average across all models of the difference between the model prediction and the average prediction

$$Bias^2 = \left(f\left(z_{i_B}^{*b}\right) - \overline{S}^* \right)^2$$
$$\widehat{Var}[S(Z)] = \frac{1}{B-1} \sum_{b=1}^B \left(S(\mathbf{Z}^{*b}) - \overline{S}^* \right)^2$$

Bias-Variance Trade-off

Measuring Bias and Variance

- Create B bootstrap variants of the dataset
- Find out the predictions for each of the B bootstrap variants
- Compute the average of all the predictions $\overline{S^*} = \frac{1}{B} \sum_{b=1}^B S(\mathbf{z}^{*b})$
- Bias: average across all models of the difference between the model prediction and the average prediction

$$Bias^2 = \sum_{i=1}^n (S(z_i^{*b}) - \overline{S^*})^2$$
$$\widehat{Var}[S(Z)] = \frac{1}{B-1} \sum_{b=1}^B (S(\mathbf{z}^{*b}) - \overline{S^*})^2$$

Bias-Variance Decomposition of Squared Error

- Assume that $y = f(\mathbf{x}) + \epsilon$
 - Noise ϵ is sampled from a normal distribution with 0 mean and variance σ^2 :
 $\epsilon \sim \mathcal{N}(0, \sigma^2)$
 - Noise lower-bounds the performance (error) we can achieve

- Recall the objective function

$$J(\boldsymbol{\theta}) = \frac{1}{2n} \sum_{i=1}^n \left(y^{(i)} - h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}) \right)^2$$

- We can represent the function as an approximation of the expected value of the squared error: $E(y - h_{\boldsymbol{\theta}}(\mathbf{x}))^2$

Bias-Variance Decomposition of Squared Error

$$y = f(x) + \epsilon$$

$$E(y - h_{\theta}(x))^2 = E \left[\underbrace{(y - f(x))}_a + \underbrace{(f(x) - h_{\theta}(x))}_b \right]^2$$

$$\begin{aligned} & E \left[(y - f(x))^2 + 2(y - f(x))(f(x) - h_{\theta}(x)) + (f(x) - h_{\theta}(x))^2 \right] \\ &= E \left[(y - f(x))^2 \right] + 2E \left[(y - f(x))(f(x) - h_{\theta}(x)) \right] + E \left[(f(x) - h_{\theta}(x))^2 \right] \\ &= E \left[\epsilon^2 \right] + \underbrace{2E \left[\epsilon (f(x) - h_{\theta}(x)) \right]}_{=0} + E \left[(f(x) - h_{\theta}(x))^2 \right] \end{aligned}$$

First term: $E \left[\epsilon^2 \right] = \text{Var}(\epsilon) + (E[\epsilon])^2 = \sigma^2$

Second term: $2E \left[\epsilon (f(x) - h_{\theta}(x)) \right] = 2E[\epsilon] E[f(x) - h_{\theta}(x)] = 0$
(Noise is uncorrelated)

3rd term: $E[(f(x) - h(x))^2]$

$$= E\left[\left(f(x) - \bar{h}(x) + (\bar{h}(x) - h(x))\right)^2\right]$$

$$\boxed{\bar{h}(x) = E[h(x)]}$$

$$= E\left[(f(x) - \bar{h}(x))^2 + 2(f(x) - \bar{h}(x))(\bar{h}(x) - h(x)) + (\bar{h}(x) - h(x))^2\right]$$

$$= E\left[(f(x) - \bar{h}(x))^2\right] + 2E\left[(f(x) - \bar{h}(x))(\bar{h}(x) - h(x))\right] + E\left[(\bar{h}(x) - h(x))^2\right]$$

$$E[\bar{h}(x) - h(x)] = 0 \Rightarrow \text{Cross term} = 0$$

$$\rightarrow \text{Bias}^2(h(x)) + E\left[(h(x) - E[h(x)])^2\right]$$

$$\therefore \text{Bias}^2(h(x)) + \text{Var}(h(x))$$

$$E[(y - h_\theta(x))^2] = \sigma^2 + \text{Bias}^2(h(x)) + \text{Var}(h(x))$$

Regularization

- Linear regression objective function

$$J(\boldsymbol{\theta}) = \frac{1}{2n} \sum_{i=1}^n (h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}) - y^{(i)})^2 + \frac{\lambda}{2} \sum_{j=1}^d \theta_j^2$$

λ is the regularization parameter.

Bias-Variance Illustration

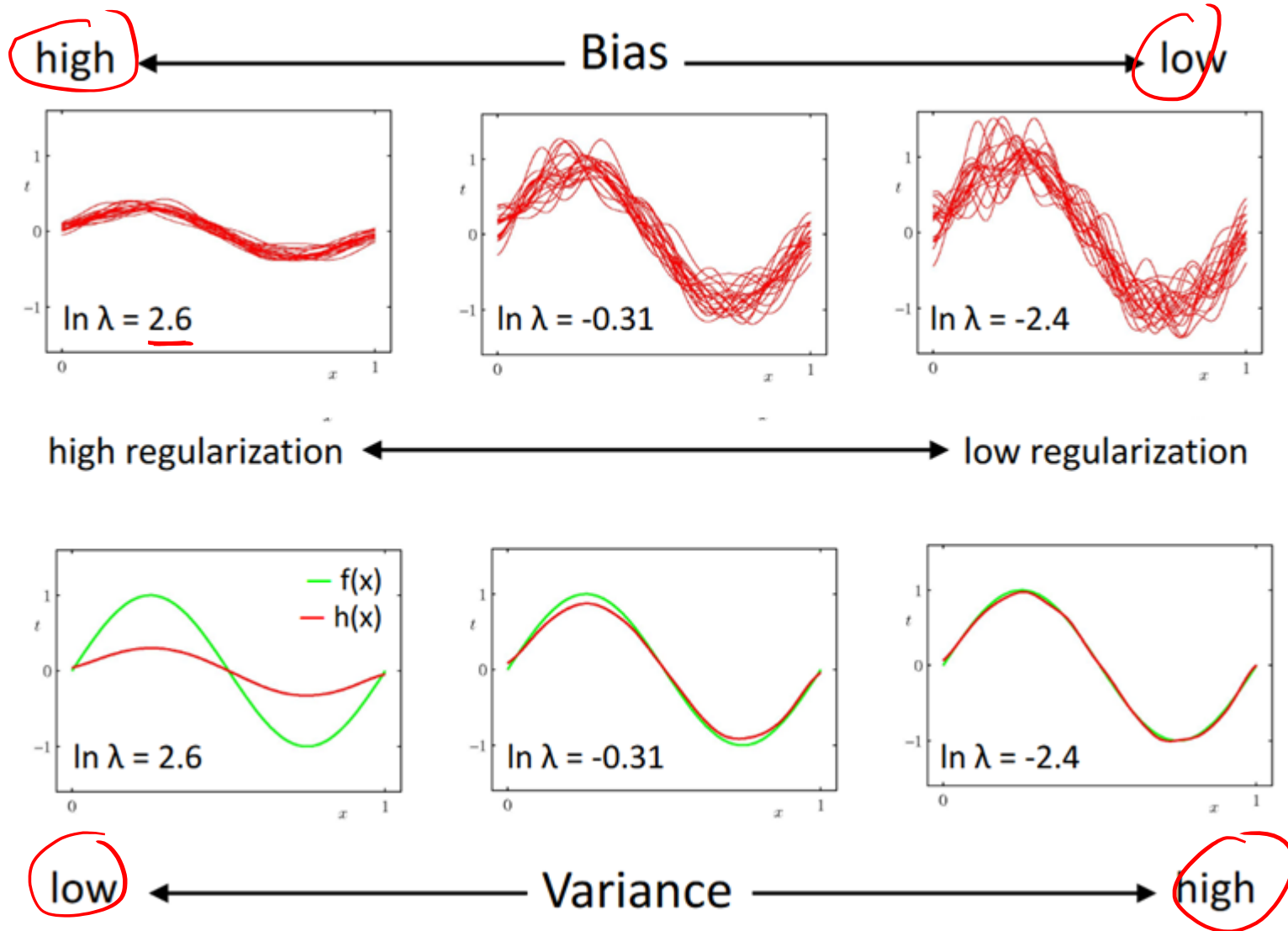


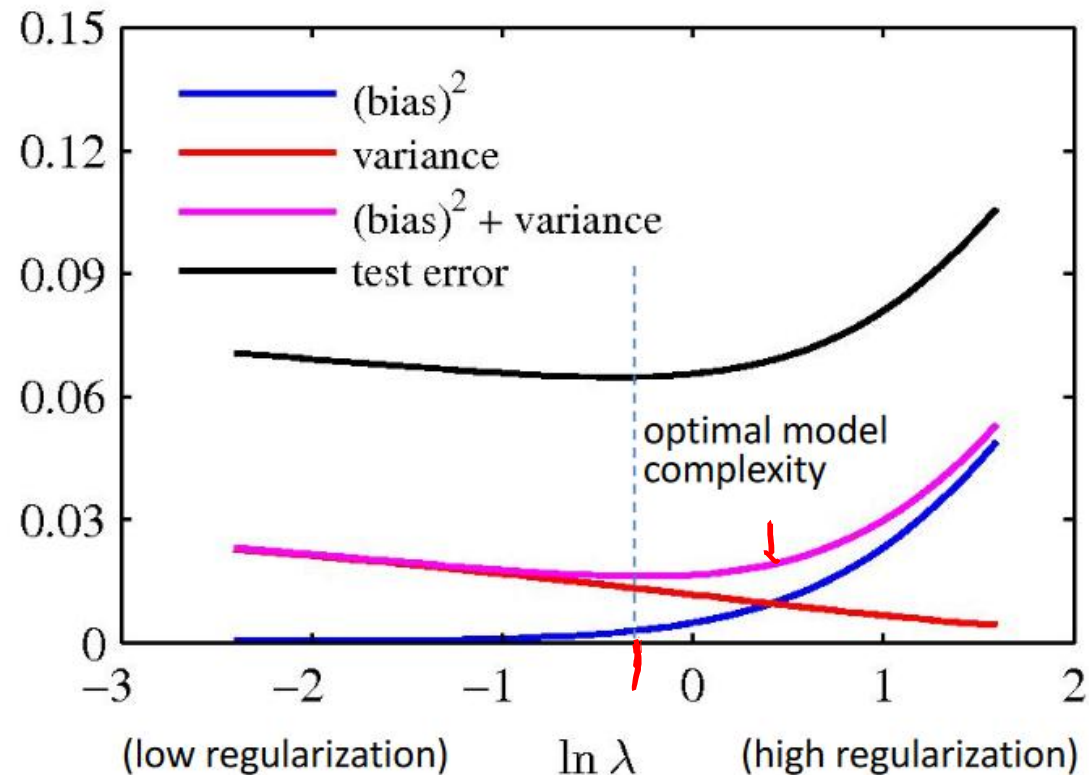
Illustration of dependence of bias and variance on model complexity (sinusoidal dataset): There are 100 datasets, each having 25 datapoints, there are 24 Gaussian basis functions in the model.

Top row: Result of fitting the model to the dataset for various values of $\ln \lambda$ (20 out of 100 fits are shown)

Bottom Row: Corresponding average of the 100 fits (red) along with the sinusoidal functions from which the dataset is generated (green).

Bias-Variance Illustration

- Reducing training error drives down bias but ignores variance



Classification

Simple Distance-Based Classifier

- Given: Labelled training data $\{(\mathbf{x}^{(1)}, y^{(1)}), (\mathbf{x}^{(2)}, y^{(2)}), \dots, (\mathbf{x}^{(n)}, y^{(n)})\}$ from two classes.
- Goal: Learn a model to predict label (class) y for a new test example $\mathbf{x}$



Simple Distance-Based Classifier

- Assume green is positive and red is negative class
- Let's say n_+ samples from positive class, n_- samples from negative class



- A simple way to assign a class is to find the distance from the mean
- This idea can be generalized to more than 2 classes as well

Distance from Means

$$\boldsymbol{\mu}_- = \frac{1}{n_-} \sum_{y_i=-1} \mathbf{x}_i \text{ and } \boldsymbol{\mu}_+ = \frac{1}{n_+} \sum_{y_i=+1} \mathbf{x}_i$$

- Once, we have the two means the training data can be discarded

$$\|\boldsymbol{\mu}_- - \mathbf{x}\|_2^2 = \|\boldsymbol{\mu}_-\|_2^2 + \|\mathbf{x}\|_2^2 - 2\boldsymbol{\mu}_-^T \mathbf{x}$$

$$\|\boldsymbol{\mu}_+ - \mathbf{x}\|_2^2 = \|\boldsymbol{\mu}_+\|_2^2 + \|\mathbf{x}\|_2^2 - 2\boldsymbol{\mu}_+^T \mathbf{x}$$

The Decision Rule

$$\begin{aligned} \mu_-^T \mu_- - 2\mu_-^T x + x^T x \\ - \mu_+^T \mu_+ + 2\mu_+^T x - x^T x \\ = \underbrace{2(\mu_+ - \mu_-)}_w^T x + b \end{aligned}$$

$$b: \|\mu_-\|_2^2 - \|\mu_+\|_2^2$$

- Let $f(x)$ be the decision rule
- $f(x) := \|\mu_- - x\|_2^2 - \|\mu_+ - x\|_2^2 = 2(\underbrace{\mu_+^T x - \mu_-^T x}_{\text{red star}}) + \|\mu_-\|_2^2 - \|\mu_+\|_2^2$

- If $f(x) > 0$ then class is +1, otherwise class is -1.

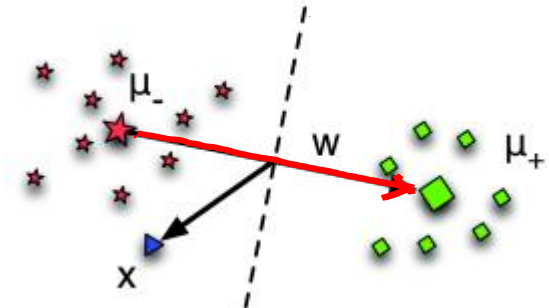
$$y = \text{sign}[f(x)]$$

$$w^T x + b = 0$$

Decision boundary

- $f(x)$ effectively denotes a hyperplane based classification rule with the vector $w = \mu_+ - \mu_-$ representing the direction normal to the hyperplane

$$\begin{aligned} x_a \quad x_b \\ w^T x_a + b = 0 \quad \text{and} \quad w^T x_b + b = 0 \\ w^T (x_a - x_b) = 0 \end{aligned}$$

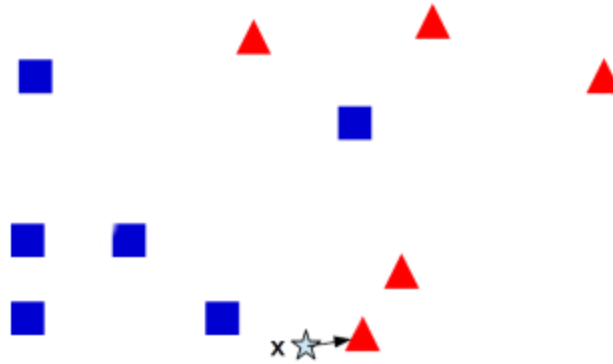


Some features of “Distance from Means”

- Simple to understand and implement
- Would usually require plenty of training data for each class (to estimate the mean of each class reliably)
- Can only learn linear decision boundaries, unless Euclidean distance is replaced by a nonlinear distance function
- Can be made more rigorous by modeling each class by a more sophisticated class probability distribution (e.g., a multivariate Gaussian) and computing distances of test points from these class distributions (rather than means)
- Several methods have this flavor, e.g., Linear (Fisher) Discriminant Analysis

Nearest Neighbor (NN) Methods

- Distance-based supervised learning method



- The label y for $x \in \mathbb{R}^d$ will be the label of its nearest neighbour in training data (a.k.a 1-NN)
- Euclidean distance can be used to find the nearest neighbor

Some Features about Nearest Neighbor

- A simple yet very effective method in practice (if given lots of training data)
 - *Provably* has an error-rate that is no worse than twice of the “Bayes optimal” classifier which assumes knowledge of the true data distribution
- Also called a memory-based or instance-based or non-parametric method
- No “model” is learned here. Prediction step uses all the training data
- Requires lots of storage (need to keep all the training data at test time)
- Prediction can be slow at test time
 - For each test point, need to compute its distance from all the training points
 - Clever data-structures or data-summarization techniques can provide speed-ups
- Need to be careful in choosing the distance function to compute distances (especially when the data dimension D is very large)
- The 1-NN can suffer if data contains outliers, or if amount of training data is small. Using more neighbors ($K > 1$) is usually more robust

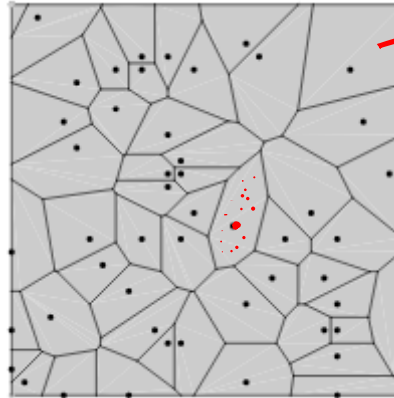
1-NN Visualization

- Every training point is a prototype.
- A test point x is assigned the label of the closest prototype
 $\hat{y}(x) = y_i$ s.t. $i = \arg \min_i \|x - x_i\|_2^2$

- 1-NN induces a Voronoi diagram (Dirichlet tessellation).

Given a set of training points
 $\{x_1, \dots, x_n\}$
 Voronoi cell of x_i

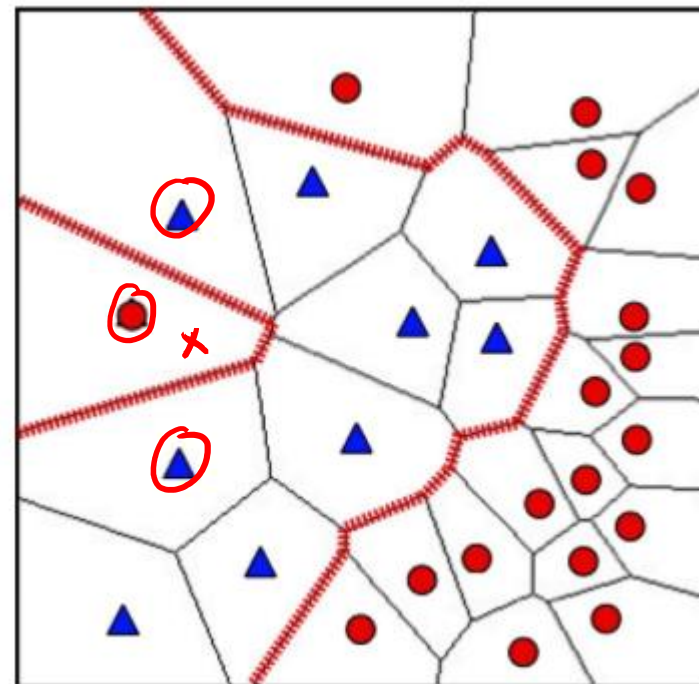
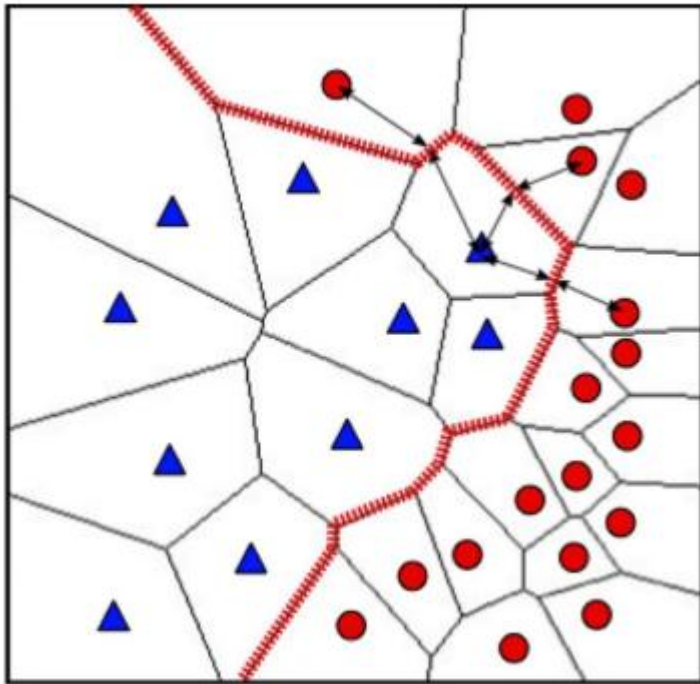
$$V_i = \left\{ x : \|x - x_i\| \leq \|x - x_j\| \forall j \neq i \right\}$$



- The partitioning of plane with n points into convex polygons
- Each polygon contains exactly one seed point
- Every point in a given polygon is closer to its seed point than to any others

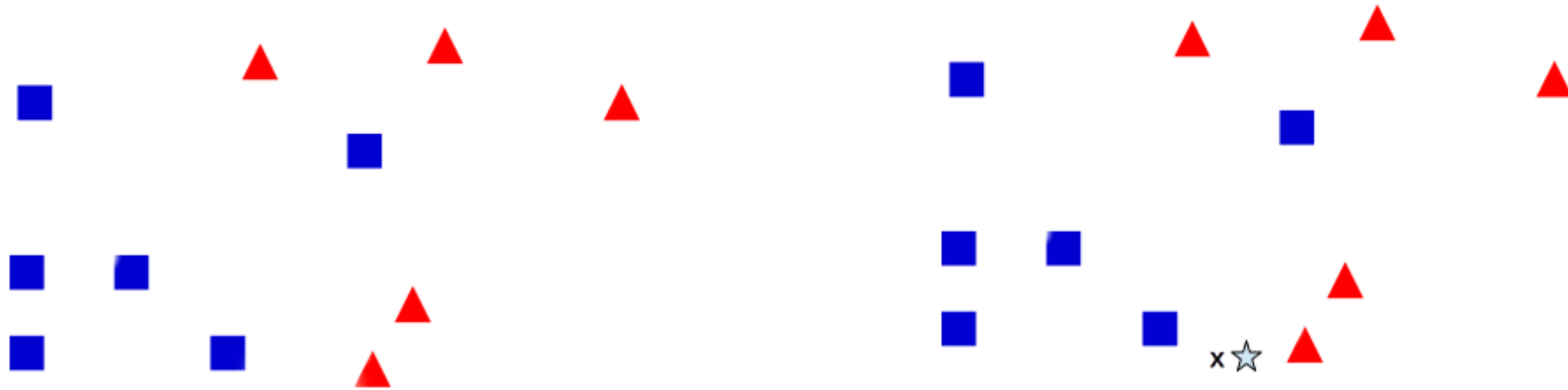
The Decision Boundary of 1-NN

- The decision boundary is composed of hyperplanes that form perpendicular bisectors of pairs of points from different classes



K - Nearest Neighbors (K -NN)

- Makes one-nearest-neighbor more robust by using more than one neighbor
- The K -NN prediction rule: Take a majority vote (or average) of the labels of $K > 1$ neighbors in the training data



K-NN Classification:-

Given training examples $\{x_i, y_i\}$

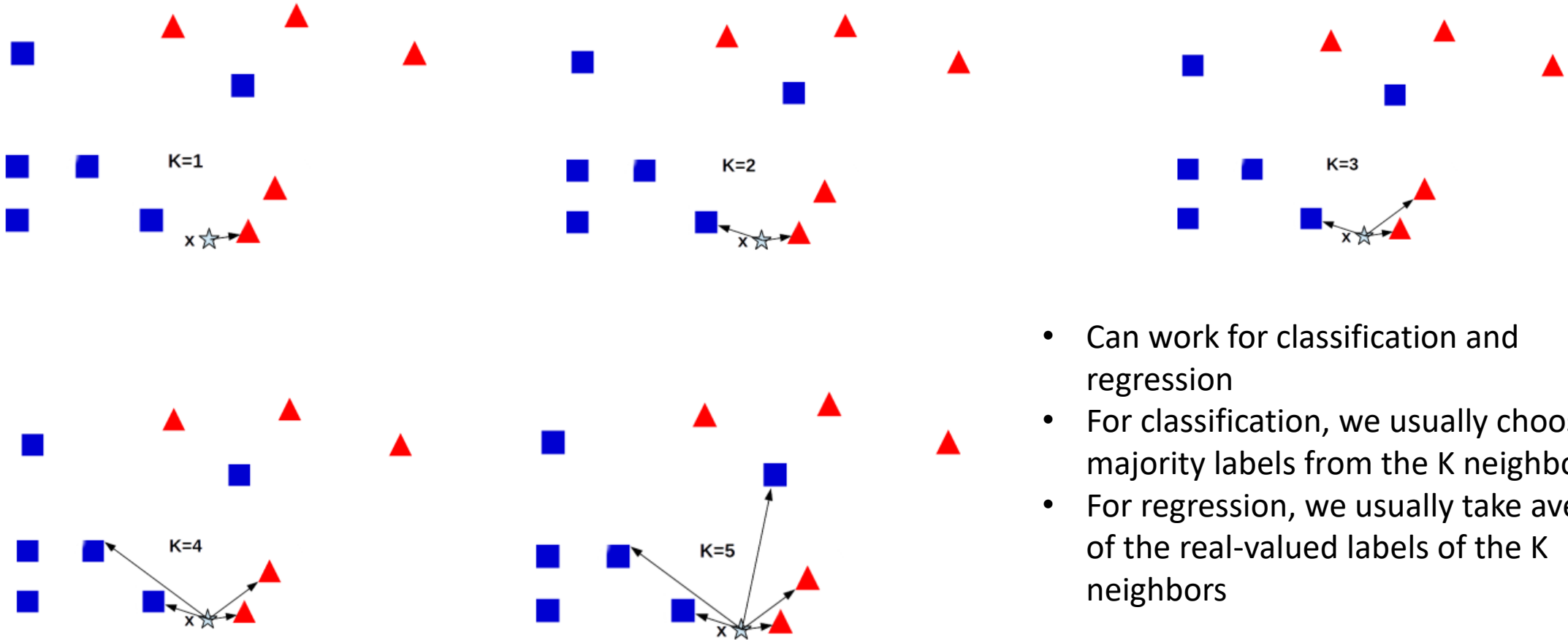
- x_i : features/examples
- y_i : label

test point x that we want to class/f.

Algorithm:-

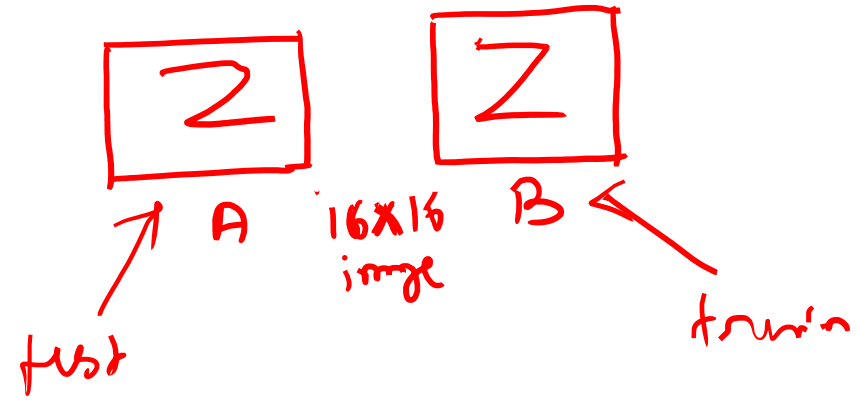
- Compute distance $D(x, x_i)$ to every training example x_i .
- Select k -closest instances $x_{i_1}, \dots, x_{i_k}$ and the labels $y_{i_1}, \dots, y_{i_k}$.
- Output the class y^* which is most frequent in $y_{i_1}, \dots, y_{i_k}$.

K - Nearest Neighbors (K -NN)



- Can work for classification and regression
- For classification, we usually choose the majority labels from the K neighbors
- For regression, we usually take average of the real-valued labels of the K neighbors

Example:- Handwritten Digit classification (mnist)



$$D(A, B) = \sqrt{\sum_{n=1}^{16} \sum_{c=1}^{16} (A_{n,c} - B_{n,c})^2}$$

FNN: ~ 95.2%

SVM: ~ 95.8%

Human: ~ 97.5%

K-NN

K-Nearest Neighbors: Decision Boundaries

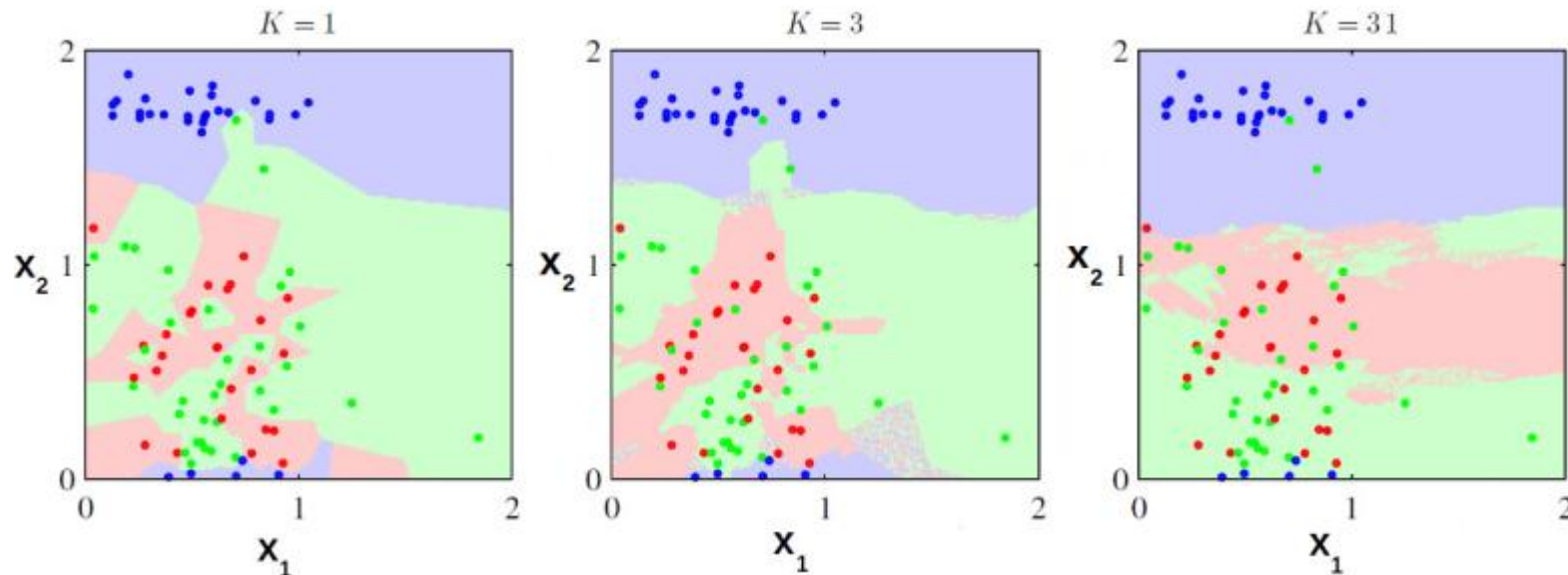
- A good value for K can be determined by considering a range of K values

- K too small: we will model the noise

- K too large: neighbors include too many points from other classes

*Very $K \rightarrow$ observe validation error
 K for which least validation error*

- Larger K leads to smoother decision boundaries

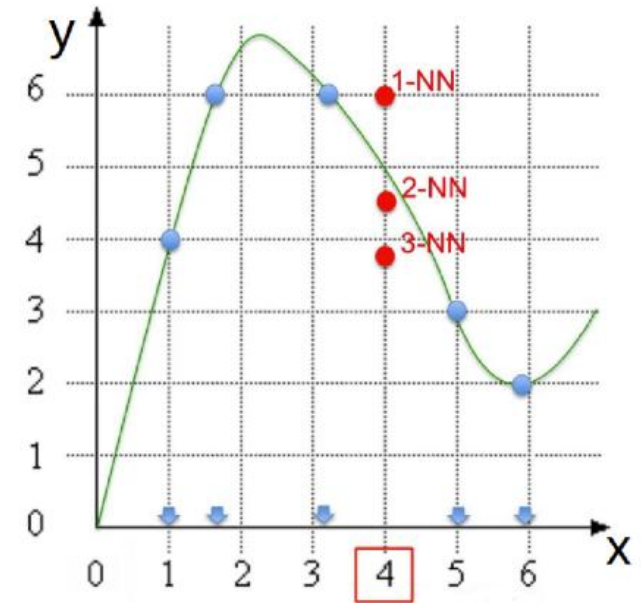


K-NN for Regression

- Given: training examples $\{(\mathbf{x}^{(1)}, y^{(1)}), (\mathbf{x}^{(2)}, y^{(2)}), \dots, (\mathbf{x}^{(n)}, y^{(n)})\}$
 - $\mathbf{x}^{(i)}$ attribute-value representation of examples
 - $y^{(i)}$ real-valued target (profit, rating on YouTube, etc.)
- Testing point $\mathbf{x}$ that we want to predict the target

Algorithm:

- compute distance $D(\mathbf{x}, \mathbf{x}^{(i)})$ to every training example
- select k closest instances $\mathbf{x}^{(1)}, \mathbf{x}^{(2)}, \dots, \mathbf{x}^{(k)}$ and their labels $y^{(1)}, y^{(2)}, \dots, y^{(k)}$
- output the mean of $y^{(1)}, y^{(2)}, \dots, y^{(k)}$: $\bar{y} = \frac{1}{k} \sum_{j=1}^k y^{(j)}$



Distance fm. -

① Euclidean Distance:- x, x'

$$D(x, x') = \sqrt{\sum_d |x_d - x'_d|^2}$$

- Symmetric, spherical, treats all dimensions equally
- Sensitive to outliers.

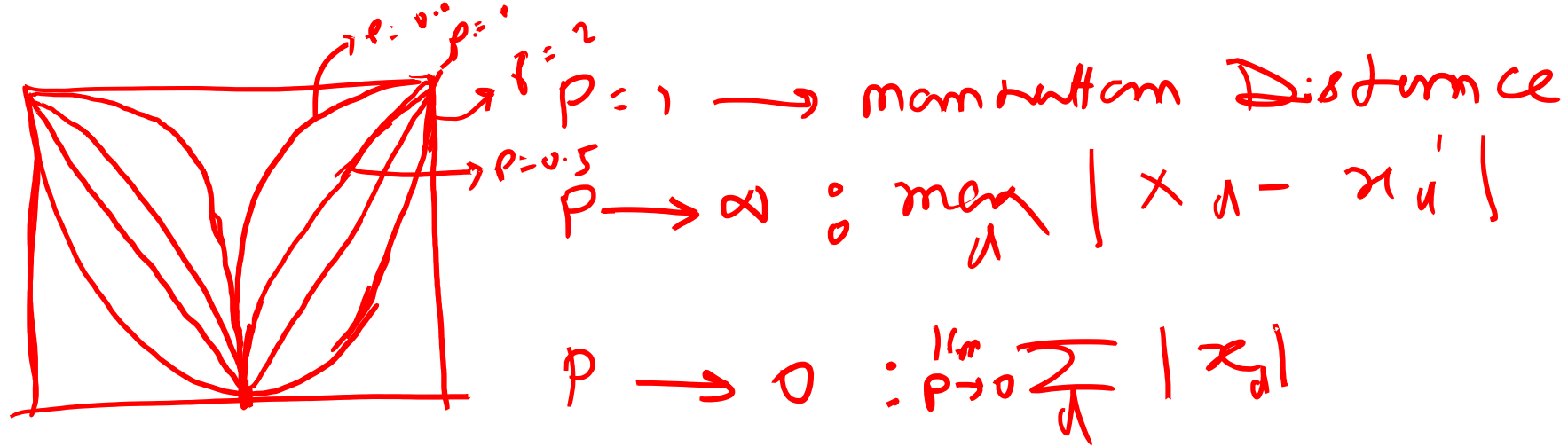
② Hamming Distance:- $D(x, x') = \sum_d \mathbb{1}_{x_d \neq x'_d}$

↳ Number of attributes where $x_d \neq x'_d$ differs

③ Minkowski Distance:- (p-norm)

$$D(x, x') = \sqrt[p]{\sum_d |x_d - x'_d|^p}$$

$p=2 \rightarrow$ Euclidean Distance



$p \leq 1 \rightarrow$ Promotes Sparsity.

(iv) Kullback - Leibler Divergence: -

For histograms ($x_d > 0, \sum_d x_d = 1$)

$$D(x, x') = - \sum_d x_d \log \frac{x_d}{x'_d}$$

Resolving Ties:-

- Equal no. of +ve / -ve neighbors.
- Breaking Rule:
 - Coin toss
 - Prior: $P(c_k) = \frac{N_k}{N}$
Choose larger $P(c_k)$
 - 1-NN

Data: $\{x_1, \dots, x_N\}$
 $\nwarrow$ $p(x)$

$\underbrace{R}_{k \text{ data points}}$ $P = \frac{k}{N}$

If R is very small $\rightarrow p(x)$ does not change much

$$\int_a^b p(x) dx \quad P \approx p(x) V \quad [\text{Volume of the region}]$$

$$\frac{k}{N} = p(x) V$$

$$p(x) = \frac{k}{N V}$$

$N_k \rightarrow C_k$ class

$$P(x | C_k) = \frac{k_k}{N_k V}$$

$$P(C_k) = \frac{N_k}{N}$$

$$P(C_k | x) = \frac{P(x | C_k) P(C_k)}{P(x)}$$

$$= \frac{\frac{k_k}{N_k V} \cdot \frac{N_k}{N}}{\frac{k}{N V}} = \frac{k_k}{k}$$

Naïve Bayes Classifier

Bayes' Rule

$$P(\text{hypothesis}|\text{evidence}) = \frac{P(\text{evidence}|\text{hypothesis}) \times P(\text{hypothesis})}{P(\text{evidence})}$$

Equivalently

$$P(Y = y_k | X = \mathbf{x}_i) = \frac{P(X = \mathbf{x}_i | Y = y_k) P(Y = y_k)}{P(X = \mathbf{x}_i)}$$

where X is a R.V. representing the evidence and Y is a R.V. for the class label (hypothesis).

$$P(Y = y_k | X = \mathbf{x}_i) = \frac{P(Y = y_k) P(X_1 = \mathbf{x}_{i,1} \cap \cdots \cap X_d = \mathbf{x}_{i,d} | Y = y_k)}{P(X_1 = \mathbf{x}_{i,1} \cap \cdots \cap X_d = \mathbf{x}_{i,d})}$$

where X_j denotes the random variable for the j th feature.

Naïve Bayes Classifier

- Idea: Use the training data to estimate $P(X|Y)$ and $P(Y)$.
- Use the Bayes' rule to infer $P(Y|X_{new})$ for new data.

$$P(Y = y_k | X = \mathbf{x}_i) = \frac{P(Y = y_k)P(X_1 = \mathbf{x}_{i,1} \cap \cdots \cap X_d = \mathbf{x}_{i,d} | Y = y_k)}{P(X_1 = \mathbf{x}_{i,1} \cap \cdots \cap X_d = \mathbf{x}_{i,d})}$$

- $P(Y = y_k)$: easy to estimate from the data
- $P(X_1 = \mathbf{x}_{i,1} \cap \cdots \cap X_d = \mathbf{x}_{i,d})$: Determination is not necessary
- $P(X_1 = \mathbf{x}_{i,1} \cap \cdots \cap X_d = \mathbf{x}_{i,d} | Y = y_k) = P(X_1, X_2, \dots, X_d | Y)$: Estimation is impractical but necessary

Naïve Bayes Classifier

- Issue: Estimating the joint PD is not practical
- If we assume that the features are independent given the class labels

$$P(X_1, X_2, \dots, X_d | Y) = \prod_{j=1}^d P(X_j | Y)$$

- We assume all the features are conditionally independent given Y
- This is often violated in practice

Training Naïve Bayes Classifier

- Estimate $P(X_j|Y)$ and $P(Y)$ from the training data by counting process.

x_1 <u>Sky</u>	x_2 <u>Temp</u>	x_3 <u>Humid</u>	x_4 <u>Wind</u>	x_5 <u>Water</u>	x_6 <u>Forecast</u>	Y <u>Play?</u>
sunny	warm	normal	strong	warm	same	yes
sunny	warm	high	strong	warm	same	yes
rainy	cold	high	strong	warm	change	no
sunny	warm	high	strong	cool	change	yes

$$\frac{\#(Sky = \text{sunny} \wedge \text{play})}{\# \text{play}}$$

$P(\text{play}) = ?$

$P(\text{Sky} = \text{sunny} \mid \text{play}) = ?$

$P(\text{Humid} = \text{high} \mid \text{play}) = ?$

...

$P(\neg \text{play}) = ?$

$P(\text{Sky} = \text{sunny} \mid \neg \text{play}) = ?$

$P(\text{Humid} = \text{high} \mid \neg \text{play}) = ?$

...

Training Naïve Bayes Classifier

- Estimate $P(X_j|Y)$ and $P(Y)$ from the training data by counting process.

<u>Sky</u>	<u>Temp</u>	<u>Humid</u>	<u>Wind</u>	<u>Water</u>	<u>Forecast</u>	<u>Play?</u>
sunny	warm	normal	strong	warm	same	yes
sunny	warm	high	strong	warm	same	yes
rainy	cold	high	strong	warm	change	no
sunny	warm	high	strong	cool	change	yes

$P(\text{play}) = ?$

$P(\text{Sky} = \text{sunny} \mid \text{play}) = ?$

$P(\text{Humid} = \text{high} \mid \text{play}) = ?$

...

$P(\neg \text{play}) = ?$

$P(\text{Sky} = \text{sunny} \mid \neg \text{play}) = ?$

$P(\text{Humid} = \text{high} \mid \neg \text{play}) = ?$

...

Training Naïve Bayes Classifier

- Estimate $P(X_j|Y)$ and $P(Y)$ from the training data by counting process.

<u>Sky</u>	<u>Temp</u>	<u>Humid</u>	<u>Wind</u>	<u>Water</u>	<u>Forecast</u>	<u>Play?</u>
sunny	warm	normal	strong	warm	same	yes
sunny	warm	high	strong	warm	same	yes
rainy	cold	high	strong	warm	change	no
sunny	warm	high	strong	cool	change	yes

$$P(\text{play}) = 3/4$$

$$P(\text{Sky} = \text{sunny} \mid \text{play}) = ?$$

$$P(\text{Humid} = \text{high} \mid \text{play}) = ?$$

...

$$P(\neg \text{play}) = 1/4$$

$$P(\text{Sky} = \text{sunny} \mid \neg \text{play}) = ?$$

$$P(\text{Humid} = \text{high} \mid \neg \text{play}) = ?$$

...

Training Naïve Bayes Classifier

- Estimate $P(X_j|Y)$ and $P(Y)$ from the training data by counting process.

<u>Sky</u>	<u>Temp</u>	<u>Humid</u>	<u>Wind</u>	<u>Water</u>	<u>Forecast</u>	<u>Play?</u>
sunny						yes
sunny						yes
rainy	cold	high	strong	warm	change	no
sunny						yes

$$P(\text{play}) = 3/4$$

$$P(\text{Sky} = \text{sunny} \mid \text{play}) = ?$$

$$P(\text{Humid} = \text{high} \mid \text{play}) = ?$$

...

$$P(\neg \text{play}) = 1/4$$

$$P(\text{Sky} = \text{sunny} \mid \neg \text{play}) = ?$$

$$P(\text{Humid} = \text{high} \mid \neg \text{play}) = ?$$

...

Training Naïve Bayes Classifier

- Estimate $P(X_j|Y)$ and $P(Y)$ from the training data by counting process.

<u>Sky</u>	<u>Temp</u>	<u>Humid</u>	<u>Wind</u>	<u>Water</u>	<u>Forecast</u>	<u>Play?</u>
sunny						yes
sunny						yes
rainy	cold	high	strong	warm	change	no
sunny						yes

$$P(\text{play}) = 3/4$$

$$P(\text{Sky} = \text{sunny} \mid \text{play}) = 1$$

$$P(\text{Humid} = \text{high} \mid \text{play}) = ?$$

...

$$P(\neg \text{play}) = 1/4$$

$$P(\text{Sky} = \text{sunny} \mid \neg \text{play}) = ?$$

$$P(\text{Humid} = \text{high} \mid \neg \text{play}) = ?$$

...

Training Naïve Bayes Classifier

- Estimate $P(X_j|Y)$ and $P(Y)$ from the training data by counting process.

<u>Sky</u>	<u>Temp</u>	<u>Humid</u>	<u>Wind</u>	<u>Water</u>	<u>Forecast</u>	<u>Play?</u>
sunny	warm	normal	strong	warm	same	yes
sunny	warm	high	strong	warm	same	yes
rainy						no
sunny	warm	high	strong	cool	change	yes

$$P(\text{play}) = 3/4$$

$$P(\text{Sky} = \text{sunny} \mid \text{play}) = 1$$

$$P(\text{Humid} = \text{high} \mid \text{play}) = ?$$

...

$$P(\neg \text{play}) = 1/4$$

$$P(\text{Sky} = \text{sunny} \mid \neg \text{play}) = ?$$

$$P(\text{Humid} = \text{high} \mid \neg \text{play}) = ?$$

...

Training Naïve Bayes Classifier

- Estimate $P(X_j|Y)$ and $P(Y)$ from the training data by counting process.

<u>Sky</u>	<u>Temp</u>	<u>Humid</u>	<u>Wind</u>	<u>Water</u>	<u>Forecast</u>	<u>Play?</u>
sunny	warm	normal	strong	warm	same	yes
sunny	warm	high	strong	warm	same	yes
rainy						no
sunny	warm	high	strong	cool	change	yes

$$P(\text{play}) = 3/4$$

$$P(\text{Sky} = \text{sunny} \mid \text{play}) = 1$$

$$P(\text{Humid} = \text{high} \mid \text{play}) = ?$$

...

$$P(\neg \text{play}) = 1/4$$

$$P(\text{Sky} = \text{sunny} \mid \neg \text{play}) = 0$$

$$P(\text{Humid} = \text{high} \mid \neg \text{play}) = ?$$

...

Training Naïve Bayes Classifier

- Estimate $P(X_j|Y)$ and $P(Y)$ from the training data by counting process.

<u>Sky</u>	<u>Temp</u>	<u>Humid</u>	<u>Wind</u>	<u>Water</u>	<u>Forecast</u>	<u>Play?</u>
		normal				yes
		high				yes
rainy	cold	high	strong	warm	change	no
		high				yes

$$P(\text{play}) = 3/4$$

$$P(\text{Sky} = \text{sunny} \mid \text{play}) = 1$$

$$P(\text{Humid} = \text{high} \mid \text{play}) = ?$$

...

$$P(\neg \text{play}) = 1/4$$

$$P(\text{Sky} = \text{sunny} \mid \neg \text{play}) = 0$$

$$P(\text{Humid} = \text{high} \mid \neg \text{play}) = ?$$

...

Training Naïve Bayes Classifier

- Estimate $P(X_j|Y)$ and $P(Y)$ from the training data by counting process.

<u>Sky</u>	<u>Temp</u>	<u>Humid</u>	<u>Wind</u>	<u>Water</u>	<u>Forecast</u>	<u>Play?</u>
		normal				yes
		high				yes
rainy	cold	high	strong	warm	change	no
		high				yes

$$P(\text{play}) = 3/4$$

$$P(\text{Sky} = \text{sunny} \mid \text{play}) = 1$$

$$P(\text{Humid} = \text{high} \mid \text{play}) = 2/3$$

...

$$P(\neg \text{play}) = 1/4$$

$$P(\text{Sky} = \text{sunny} \mid \neg \text{play}) = 0$$

$$P(\text{Humid} = \text{high} \mid \neg \text{play}) = ?$$

...

Training Naïve Bayes Classifier

- Estimate $P(X_j|Y)$ and $P(Y)$ from the training data by counting process.

<u>Sky</u>	<u>Temp</u>	<u>Humid</u>	<u>Wind</u>	<u>Water</u>	<u>Forecast</u>	<u>Play?</u>
sunny	warm	normal	strong	warm	same	yes
sunny	warm	high	strong	warm	same	yes
		high				no
sunny	warm	high	strong	cool	change	yes

$$P(\text{play}) = 3/4$$

$$P(\text{Sky} = \text{sunny} \mid \text{play}) = 1$$

$$P(\text{Humid} = \text{high} \mid \text{play}) = 2/3$$

...

$$P(\neg \text{play}) = 1/4$$

$$P(\text{Sky} = \text{sunny} \mid \neg \text{play}) = 0$$

$$P(\text{Humid} = \text{high} \mid \neg \text{play}) = ?$$

...

Training Naïve Bayes Classifier

- Estimate $P(X_j|Y)$ and $P(Y)$ from the training data by counting process.

<u>Sky</u>	<u>Temp</u>	<u>Humid</u>	<u>Wind</u>	<u>Water</u>	<u>Forecast</u>	<u>Play?</u>
sunny	warm	normal	strong	warm	same	yes
sunny	warm	high	strong	warm	same	yes
		high				no
sunny	warm	high	strong	cool	change	yes

$$P(\text{play}) = 3/4$$

$$P(\text{Sky} = \text{sunny} \mid \text{play}) = 1$$

$$P(\text{Humid} = \text{high} \mid \text{play}) = 2/3$$

...

$$P(\neg \text{play}) = 1/4$$

$$P(\text{Sky} = \text{sunny} \mid \neg \text{play}) = 0$$

$$P(\text{Humid} = \text{high} \mid \neg \text{play}) = 1$$

...

Laplace Smoothing

- Some probabilities estimated by counting process might be zero—possible case of overfitting!

- Regularize by Laplace smoothing

- Add 1 to each count

$$P(X_j = v | Y = y_k) = \frac{c_v + 1}{\sum_{v' \in \text{values}(X_j)} c_{v'} + |\text{values}(X_j)|}$$

- c_v is the count of training examples with a value of v for feature j and class label y_k
 - $\sum_{v' \in \text{values}(X_j)} c_{v'}$ = Count of values in that class
 - $|\text{values}(X_j)|$ is the number of values X_j can take on

$$\frac{c_v}{\sum_{v'} c_{v'}}$$

Training Naïve Bayes with Laplace Smoothing

- Estimate $P(X_j|Y)$ and $P(Y)$ from the training data by counting with Laplace smoothing

<u>Sky</u>	<u>Temp</u>	<u>Humid</u>	<u>Wind</u>	<u>Water</u>	<u>Forecast</u>	<u>Play?</u>
sunny						yes
sunny						yes
rainy	cold	high	strong	warm	change	no
sunny						yes

$$\frac{3+1}{3+2} = \frac{4}{5}$$

$$P(\text{play}) = 3/4$$

$$P(\neg \text{play}) = 1/4$$

$$P(\text{Sky} = \text{sunny} \mid \text{play}) = 4/5$$

$$P(\text{Sky} = \text{sunny} \mid \neg \text{play}) = ?$$

$$\frac{0+1}{1+2} = \frac{1}{3}$$

$$P(\text{Humid} = \text{high} \mid \text{play}) = ?$$

$$P(\text{Humid} = \text{high} \mid \neg \text{play}) = ?$$

...

...

Training Naïve Bayes with Laplace Smoothing

- Estimate $P(X_j|Y)$ and $P(Y)$ from the training data by counting with Laplace smoothing

<u>Sky</u>	<u>Temp</u>	<u>Humid</u>	<u>Wind</u>	<u>Water</u>	<u>Forecast</u>	<u>Play?</u>
sunny	warm	normal	strong	warm	same	yes
sunny	warm	high	strong	warm	same	yes
rainy						no
sunny	warm	high	strong	cool	change	yes

$$P(\text{play}) = 3/4$$

$$P(\neg \text{play}) = 1/4$$

$$P(\text{Sky} = \text{sunny} \mid \text{play}) = 4/5 \quad P(\text{Sky} = \text{sunny} \mid \neg \text{play}) = 1/3$$

$$P(\text{Humid} = \text{high} \mid \text{play}) = ? \quad P(\text{Humid} = \text{high} \mid \neg \text{play}) = ?$$

...

...

Training Naïve Bayes with Laplace Smoothing

- Estimate $P(X_j|Y)$ and $P(Y)$ from the training data by counting with Laplace smoothing

<u>Sky</u>	<u>Temp</u>	<u>Humid</u>	<u>Wind</u>	<u>Water</u>	<u>Forecast</u>	<u>Play?</u>
		normal				yes
		high				yes
rainy	cold	high	strong	warm	change	no
		high				yes

$$P(\text{play}) = 3/4$$

$$P(\neg \text{play}) = 1/4$$

$$P(\text{Sky} = \text{sunny} \mid \text{play}) = 4/5 \quad P(\text{Sky} = \text{sunny} \mid \neg \text{play}) = 1/3$$

$$P(\text{Humid} = \text{high} \mid \text{play}) = 3/5 \quad P(\text{Humid} = \text{high} \mid \neg \text{play}) = ?$$

...

...

Training Naïve Bayes with Laplace Smoothing

- Estimate $P(X_j|Y)$ and $P(Y)$ from the training data by counting with Laplace smoothing

<u>Sky</u>	<u>Temp</u>	<u>Humid</u>	<u>Wind</u>	<u>Water</u>	<u>Forecast</u>	<u>Play?</u>
sunny	warm	normal	strong	warm	same	yes
sunny	warm	high	strong	warm	same	yes
		high				no
sunny	warm	high	strong	cool	change	yes

$$P(\text{play}) = 3/4$$

$$P(\neg \text{play}) = 1/4$$

$$P(\text{Sky} = \text{sunny} \mid \text{play}) = 4/5 \quad P(\text{Sky} = \text{sunny} \mid \neg \text{play}) = 1/3$$

$$P(\text{Humid} = \text{high} \mid \text{play}) = 3/5 \quad P(\text{Humid} = \text{high} \mid \neg \text{play}) = 2/3$$

...

...

Using the Classifier

- We have $P(Y = y_k | X = \mathbf{x}_i) = \frac{P(X=\mathbf{x}_i|Y=y_k)P(Y=y_k)}{P(X=\mathbf{x}_i)}$
- $P(X = \mathbf{x}_i)$ is a constant for a given example (not required for prediction)
- To classify a new point $\mathbf{x}$

$$h(\mathbf{x}) = \arg \max_{y_k} P(Y = y_k) \prod_{j=1}^d P(X_j = \mathbf{x}_j | Y = y_k)$$

$$h(\mathbf{x}) = \arg \max_{y_k} \log P(Y = y_k) + \sum_{j=1}^d \log P(X_j = \mathbf{x}_j | Y = y_k)$$

The Naïve Bayes Classifier Algorithm

- For each class y_k
 - Estimate $P(Y = y_k)$ from the data
 - For each value $x_{i,j}$ of each feature X_i -- Estimate $P(X_i = x_{i,j} | Y = y_k)$
- Classify a new point using:

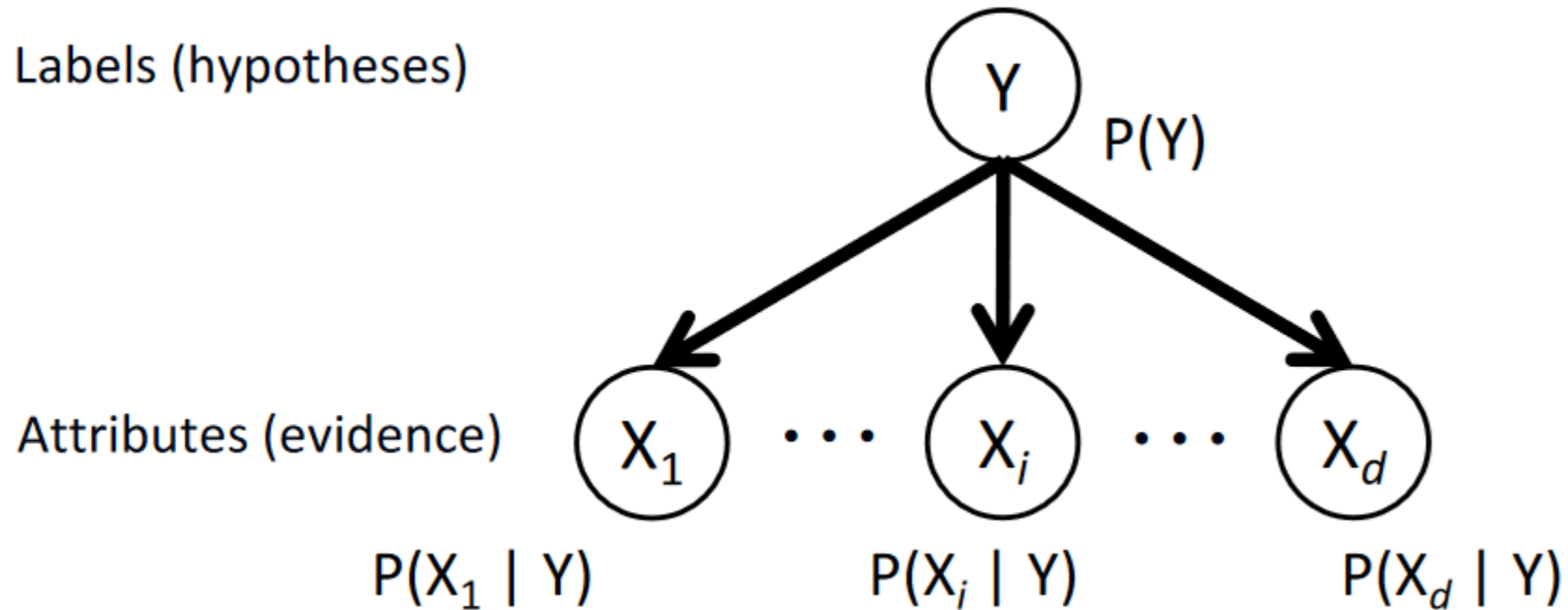
~~Tricky~~

$$h(x) = \arg \max_{y_k} \log P(Y = y_k) + \sum_{j=1}^d \log P(X_j = \mathbf{x}_j | Y = y_k)$$

- Although the independence assumption does not often hold true, Naïve Bayes Classifier performs quite well.

Graphical Model

Labels (hypotheses)

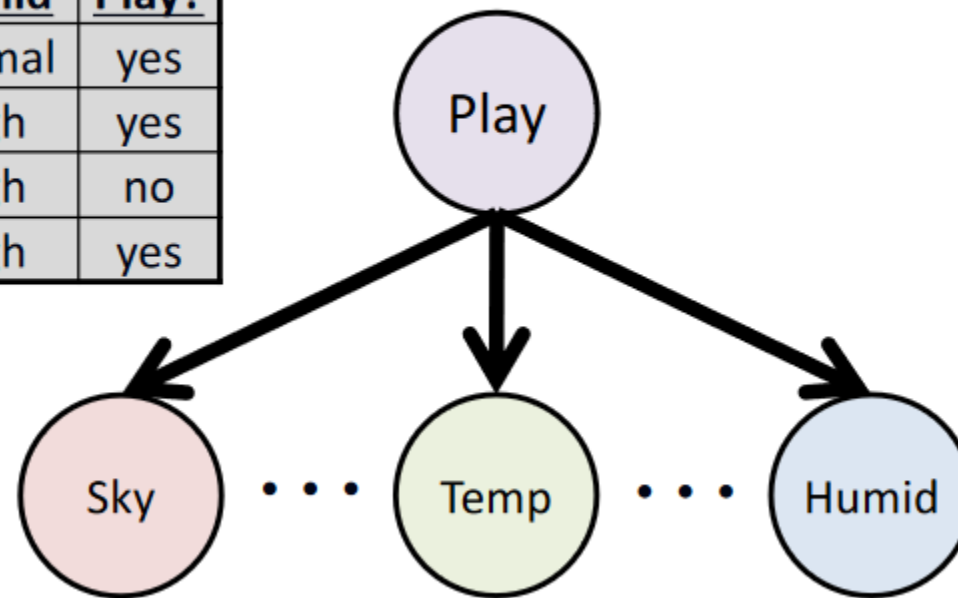


- Nodes denote random variables
- Edges denote dependency
- Each node is associated to its parents via conditional probability

Example

Data:

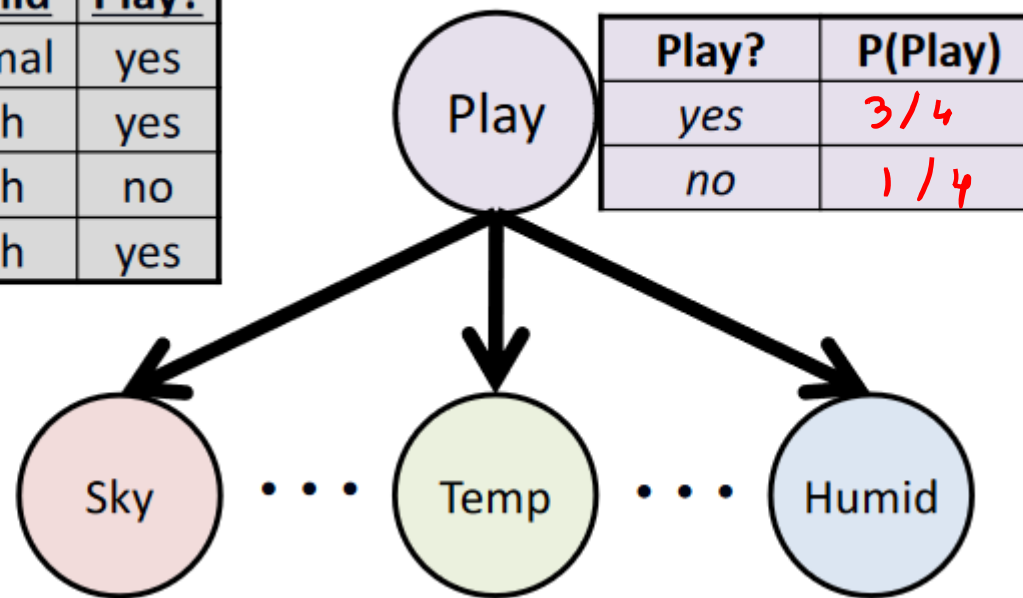
<u>Sky</u>	<u>Temp</u>	<u>Humid</u>	<u>Play?</u>
sunny	warm	normal	yes
sunny	warm	high	yes
rainy	cold	high	no
sunny	warm	high	yes



Example

Data:

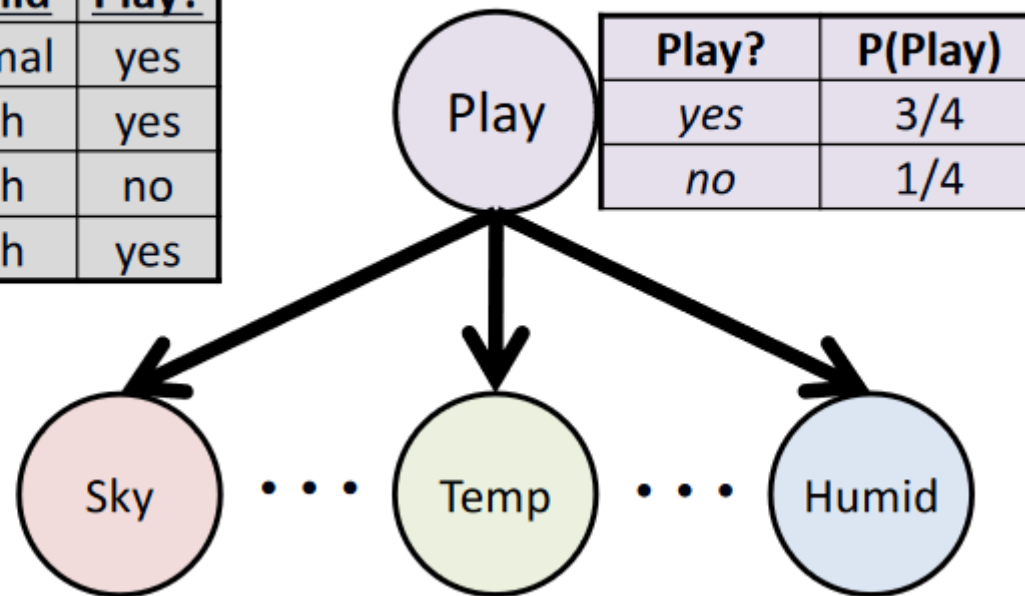
<u>Sky</u>	<u>Temp</u>	<u>Humid</u>	<u>Play?</u>
sunny	warm	normal	yes
sunny	warm	high	yes
rainy	cold	high	no
sunny	warm	high	yes



Example

Data:

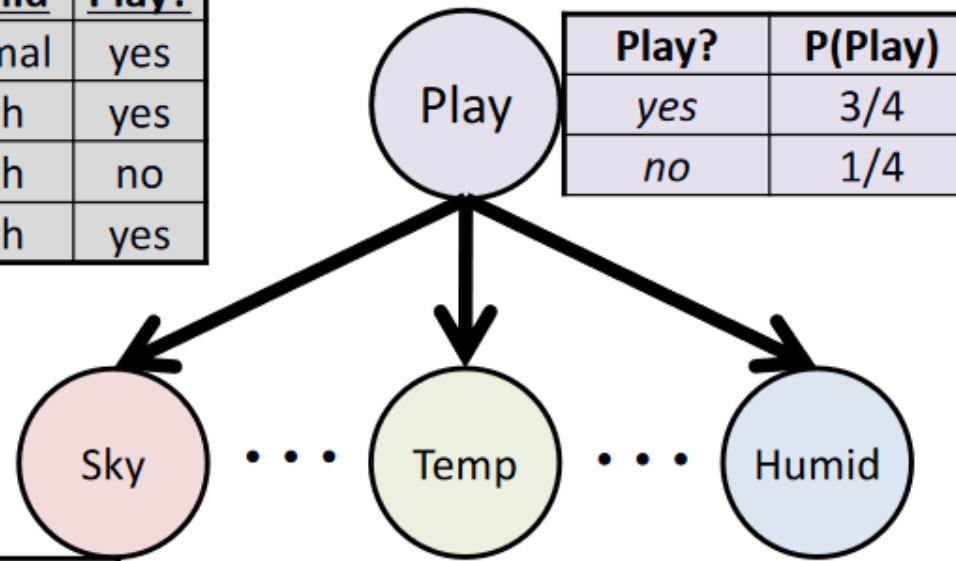
<u>Sky</u>	<u>Temp</u>	<u>Humid</u>	<u>Play?</u>
sunny	warm	normal	yes
sunny	warm	high	yes
rainy	cold	high	no
sunny	warm	high	yes



Example

Data:

<u>Sky</u>	<u>Temp</u>	<u>Humid</u>	<u>Play?</u>
sunny	warm	normal	yes
sunny	warm	high	yes
rainy	cold	high	no
sunny	warm	high	yes



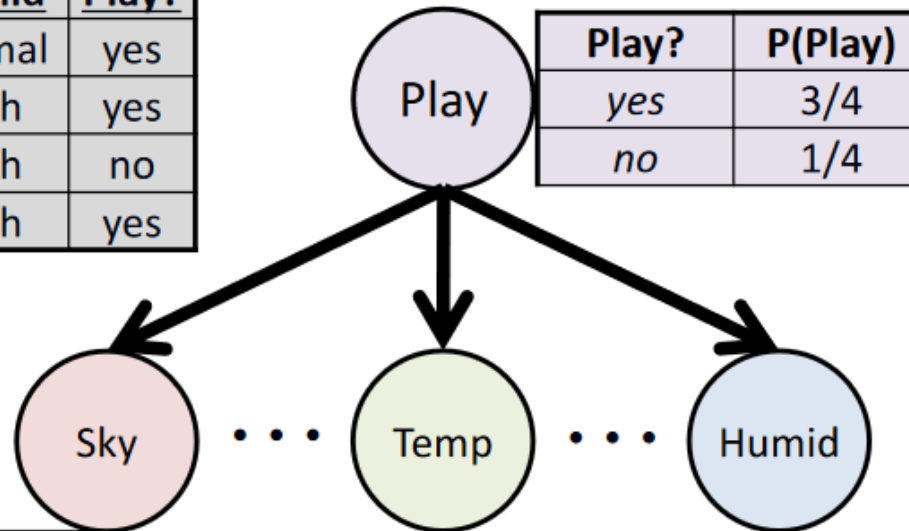
<u>Play?</u>	<u>P(Play)</u>
<i>yes</i>	3/4
<i>no</i>	1/4

<u>Sky</u>	<u>Play?</u>	<u>P(Sky Play)</u>
<i>sunny</i>	<i>yes</i>	<i>4/5</i>
<i>rainy</i>	<i>yes</i>	<i>1/5</i>
<i>sunny</i>	<i>no</i>	<i>1/3</i>
<i>rainy</i>	<i>no</i>	<i>2/3</i>

Example

Data:

<u>Sky</u>	<u>Temp</u>	<u>Humid</u>	<u>Play?</u>
sunny	warm	normal	yes
sunny	warm	high	yes
rainy	cold	high	no
sunny	warm	high	yes



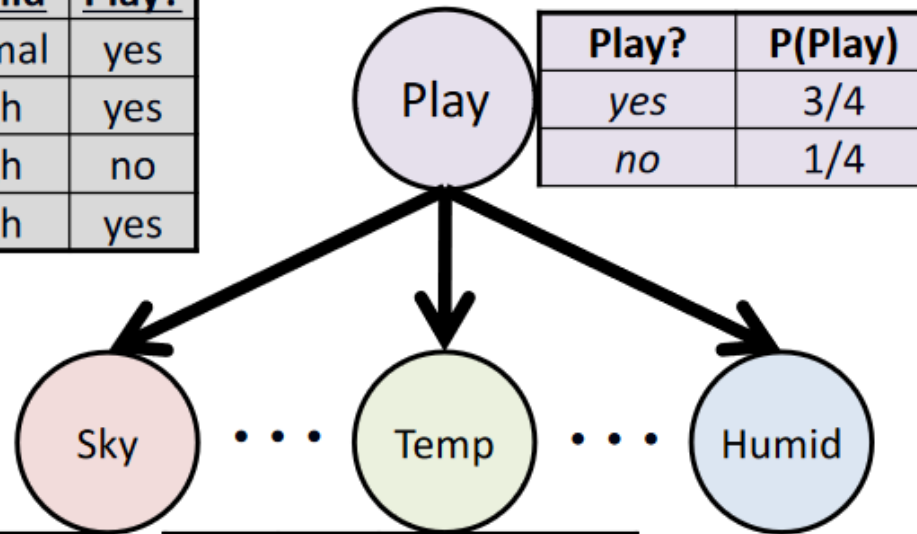
Play?	P(Play)
<i>yes</i>	3/4
<i>no</i>	1/4

Sky	Play?	P(Sky Play)
<i>sunny</i>	<i>yes</i>	4/5
<i>rainy</i>	<i>yes</i>	1/5
<i>sunny</i>	<i>no</i>	1/3
<i>rainy</i>	<i>no</i>	2/3

Example

Data:

<u>Sky</u>	<u>Temp</u>	<u>Humid</u>	<u>Play?</u>
sunny	warm	normal	yes
sunny	warm	high	yes
rainy	cold	high	no
sunny	warm	high	yes



<u>Play?</u>	<u>P(Play)</u>
yes	3/4
no	1/4

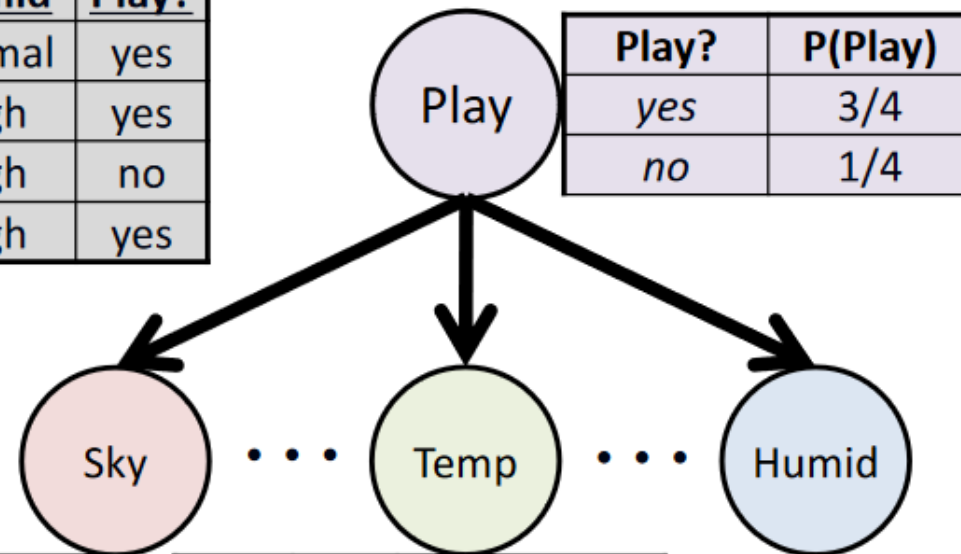
<u>Sky</u>	<u>Play?</u>	<u>P(Sky Play)</u>
sunny	yes	4/5
rainy	yes	1/5
sunny	no	1/3
rainy	no	2/3

<u>Temp</u>	<u>Play?</u>	<u>P(Temp Play)</u>
warm	yes	4/5
cold	yes	1/5
warm	no	1/3
cold	no	2/3

Example

Data:

<u>Sky</u>	<u>Temp</u>	<u>Humid</u>	<u>Play?</u>
sunny	warm	normal	yes
sunny	warm	high	yes
rainy	cold	high	no
sunny	warm	high	yes



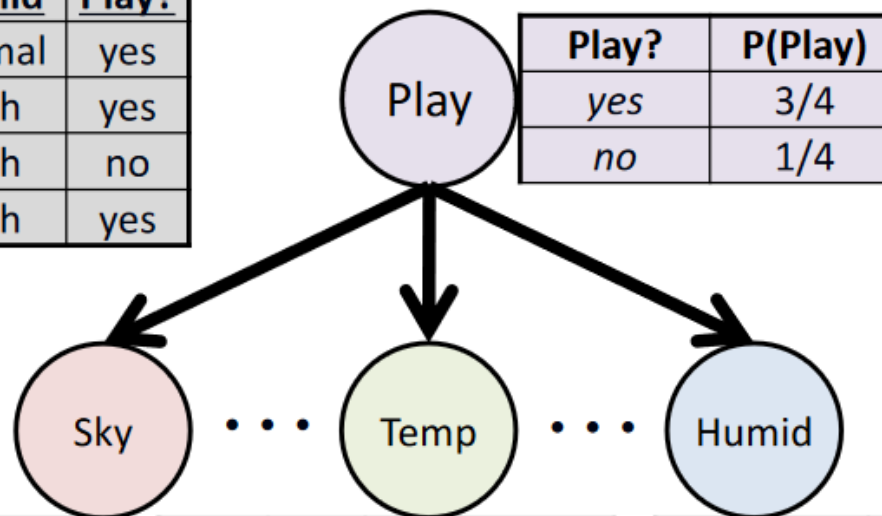
<u>Sky</u>	<u>Play?</u>	<u>P(Sky Play)</u>
sunny	yes	4/5
rainy	yes	1/5
sunny	no	1/3
rainy	no	2/3

<u>Temp</u>	<u>Play?</u>	<u>P(Temp Play)</u>
warm	yes	4/5
cold	yes	1/5
warm	no	1/3
cold	no	2/3

Example

Data:

Sky	Temp	Humid	Play?
sunny	warm	normal	yes
sunny	warm	high	yes
rainy	cold	high	no
sunny	warm	high	yes



Play?	P(Play)
yes	3/4
no	1/4

Sky	Play?	P(Sky Play)
sunny	yes	4/5
rainy	yes	1/5
sunny	no	1/3
rainy	no	2/3

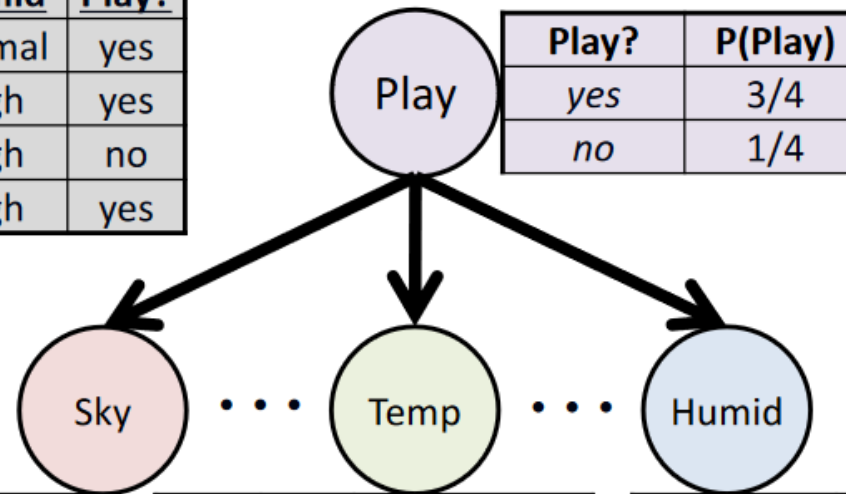
Temp	Play?	P(Temp Play)
warm	yes	4/5
cold	yes	1/5
warm	no	1/3
cold	no	2/3

Humid	Play?	P(Humid Play)
high	yes	3/5
norm	yes	2/5
high	no	2/3
norm	no	1/3

Example

Data:

<u>Sky</u>	<u>Temp</u>	<u>Humid</u>	<u>Play?</u>
sunny	warm	normal	yes
sunny	warm	high	yes
rainy	cold	high	no
sunny	warm	high	yes



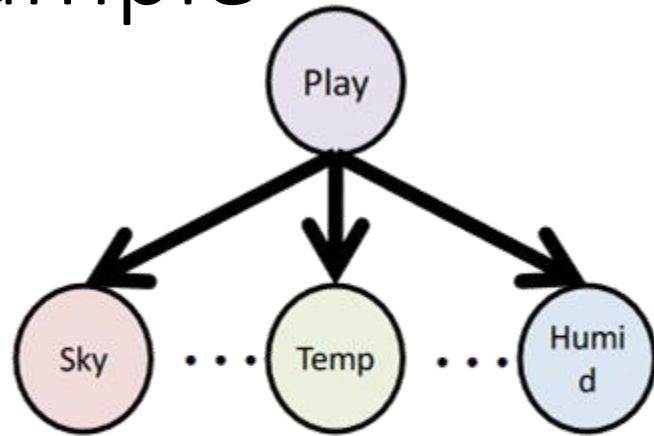
<u>Play?</u>	<u>P(Play)</u>
<i>yes</i>	3/4
<i>no</i>	1/4

<u>Sky</u>	<u>Play?</u>	<u>P(Sky Play)</u>
<i>sunny</i>	<i>yes</i>	4/5
<i>rainy</i>	<i>yes</i>	1/5
<i>sunny</i>	<i>no</i>	1/3
<i>rainy</i>	<i>no</i>	2/3

<u>Temp</u>	<u>Play?</u>	<u>P(Temp Play)</u>
<i>warm</i>	<i>yes</i>	4/5
<i>cold</i>	<i>yes</i>	1/5
<i>warm</i>	<i>no</i>	1/3
<i>cold</i>	<i>no</i>	2/3

<u>Humid</u>	<u>Play?</u>	<u>P(Humid Play)</u>
<i>high</i>	<i>yes</i>	3/5
<i>norm</i>	<i>yes</i>	2/5
<i>high</i>	<i>no</i>	2/3
<i>norm</i>	<i>no</i>	1/3

Example



Play?	P(Play)
<i>yes</i>	3/4
<i>no</i>	1/4

Temp	Play?	P(Temp Play)
<i>warm</i>	<i>yes</i>	4/5
<i>cold</i>	<i>yes</i>	1/5
<i>warm</i>	<i>no</i>	1/3
<i>cold</i>	<i>no</i>	2/3

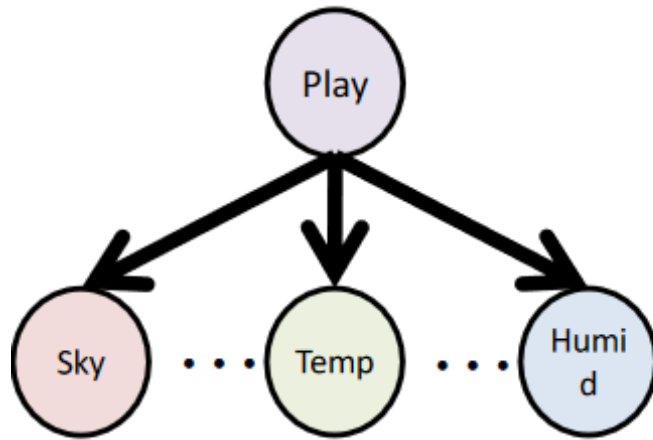
Sky	Play?	P(Sky Play)
<i>sunny</i>	<i>yes</i>	4/5
<i>rainy</i>	<i>yes</i>	1/5
<i>sunny</i>	<i>no</i>	1/3
<i>rainy</i>	<i>no</i>	2/3

Humid	Play?	P(Humid Play)
<i>high</i>	<i>yes</i>	3/5
<i>norm</i>	<i>yes</i>	2/5
<i>high</i>	<i>no</i>	2/3
<i>norm</i>	<i>no</i>	1/3

$$h(x) = \arg \max_{y_k} \log P(Y = y_k) + \sum_{j=1}^d \log P(X_j = x_j | Y = y_k)$$

Objective: Predict label for $x = (rainy, warm, normal)$

Example



Predict label for:
 $\mathbf{x} = (\text{rainy}, \text{warm}, \text{normal})$

Play?	P(Play)
yes	3/4
no	1/4

Temp	Play?	P(Temp Play)
warm	yes	4/5
cold	yes	1/5
warm	no	1/3
cold	no	2/3

Sky	Play?	P(Sky Play)
sunny	yes	4/5
rainy	yes	1/5
sunny	no	1/3
rainy	no	2/3

Humid	Play?	P(Humid Play)
high	yes	3/5
norm	yes	2/5
high	no	2/3
norm	no	1/3

$$\begin{aligned}
 P(\text{play} \mid \mathbf{x}) &\propto \log P(\text{play}) + \log P(\text{rainy} \mid \text{play}) + \log P(\text{warm} \mid \text{play}) + \log P(\text{normal} \mid \text{play}) \\
 &\propto \log 3/4 + \log 1/5 + \log 4/5 + \log 2/5 = -1.319 \quad \text{predict PLAY}
 \end{aligned}$$

$$\begin{aligned}
 P(\neg \text{play} \mid \mathbf{x}) &\propto \log P(\neg \text{play}) + \log P(\text{rainy} \mid \neg \text{play}) + \log P(\text{warm} \mid \neg \text{play}) + \log P(\text{normal} \mid \neg \text{play}) \\
 &\propto \log 1/4 + \log 2/3 + \log 1/3 + \log 1/3 = -1.732
 \end{aligned}$$

Summary

- **Advantages**

- Fast training
- Fast classification
- Not sensitive to irrelevant features
- Can handle real-valued data

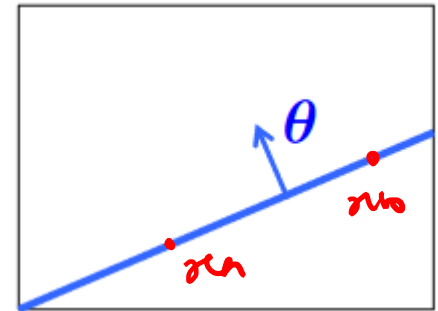
- **Disadvantage:**

- Assume independence of features

Linear Classification: The Perceptron

Linear Classifiers

- A hyperplane partitions $\mathbb{R}^d$ into two half-spaces, defined by the normal vector $\theta \in \mathbb{R}^d$
 - θ is orthogonal to any vector lying on the hyperplane
 - Assumed to pass through the origin as we have incorporated bias term θ_0 into it by $x_0 = 1$
- Consider classification with +1 and -1 labels



$$\theta^T x + b = 0$$

$$\theta^T x_a + b = 0$$

$$\theta^T x_b + b = 0$$

$$\theta^T (x_a - x_b) = 0$$

Linear Classifiers

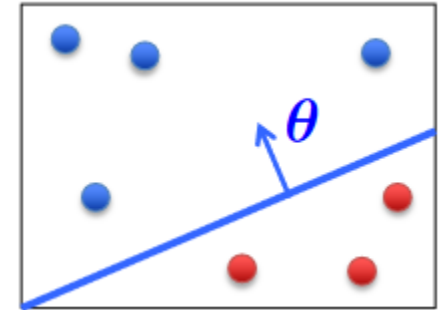
- Represent decision boundary by hyperplane

- $\boldsymbol{\theta} = \begin{bmatrix} \theta_0 \\ \theta_1 \\ \vdots \\ \theta_d \end{bmatrix} \quad \mathbf{x}^T = [1 \quad x_1 \quad \cdots \quad x_d]$

- $h(\mathbf{x}) = \text{sign}(\boldsymbol{\theta}^T \mathbf{x})$ where $\text{sign}(z) = \begin{cases} 1 & \text{if } z \geq 0 \\ -1 & \text{if } z < 0 \end{cases}$

- $\boldsymbol{\theta}^T \mathbf{x} > 0 \Rightarrow y = +1$

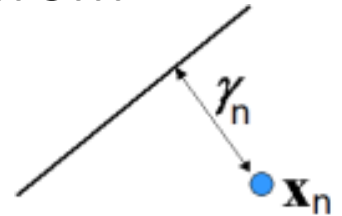
- $\boldsymbol{\theta}^T \mathbf{x} < 0 \Rightarrow y = -1$



Notion of Margins

- Geometric Margin $\gamma^{(i)}$ of an example $\mathbf{x}^{(i)}$ is its signed distance from hyperplane

$$\gamma^{(i)} = \frac{\boldsymbol{\theta}^T \mathbf{x}^{(i)}}{\|\boldsymbol{\theta}\|}$$



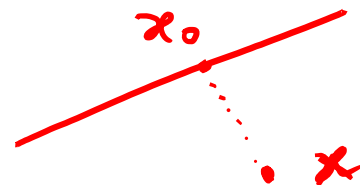
- $\gamma^{(i)}$ may be +ve or -ve based on which side of the plane $\mathbf{x}^{(i)}$ is
- Margin of a set $\{\mathbf{x}^{(1)}, \mathbf{x}^{(2)}, \dots, \mathbf{x}^{(n)}\}$ with respect to $\boldsymbol{\theta}$ is the minimum absolute geometric margin

$$\gamma = \min_{1 \leq i \leq n} |\gamma^{(i)}| = \min_{1 \leq i \leq n} \frac{|\boldsymbol{\theta}^T \mathbf{x}^{(i)}|}{\|\boldsymbol{\theta}\|}$$

- Functional margin of $\boldsymbol{\theta}$ on a training example $(\mathbf{x}^{(i)}, y^{(i)})$: $y^{(i)}(\boldsymbol{\theta}^T \mathbf{x}^{(i)})$
 - +ve for correct prediction
 - ve for incorrect prediction

margin :-

$$\theta^T x + b = 0 \quad \text{--- Hyperplane}$$



$\frac{\theta}{\|\theta\|}$ = unit length normal vector

$(x - x_0) \perp \text{Hyperplane}$

$$x = x_0 + t \frac{\theta}{\|\theta\|}$$

[t = Signed Distance]

$$\theta^T x_0 + b = 0 \quad [\because x_0 \text{ lies on the hyperplane}]$$

$$\theta^T x + b$$

$$= \theta^T \left(x_0 + t \frac{\theta}{\|\theta\|} \right) + b = \theta^T x_0 + t \frac{\theta^T \theta}{\|\theta\|} + b$$

$$= \underbrace{\theta^T x_0 + b}_{=0} + t \frac{\|\theta\|^2}{\|\theta\|} = t \|\theta\|$$

$$t = \frac{\theta^T x + b}{\|\theta\|} \quad [t = \gamma]$$

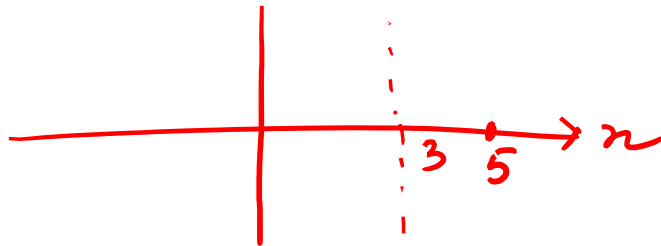
$$\boxed{\gamma^{(i)} = \frac{\theta^T x^{(i)} + b}{\|\theta\|}}$$

$$\gamma = t = \frac{0^T x + b}{\|a\|}$$

#

$$2x - 6 = 0$$

$$x = 3$$



$x = 5 \Rightarrow |5 - 3| = 2$ is distance x and the decision boundary.

$$\gamma = \frac{|2(5) - 6|}{2} = \frac{4}{2} = 2$$

$$20x - 60 = 0$$

$$x = 5 \Rightarrow 20(5) - 60 = 40 \text{ for value}$$

$$\gamma = \frac{20(5) - 60}{20} = 2$$

Loss function for linear classifier

- For a hyperplane based model

$$J(\boldsymbol{\theta}) = \frac{1}{n} \sum_{i=1}^n \max(0, -y^{(i)}(\boldsymbol{\theta}^T \mathbf{x}^{(i)}))$$

- If $y^{(i)}(\boldsymbol{\theta}^T \mathbf{x}^{(i)}) \geq 0$, then $\max(0, -y^{(i)}(\boldsymbol{\theta}^T \mathbf{x}^{(i)}))$ is zero for correct prediction
- Otherwise, $y^{(i)}(\boldsymbol{\theta}^T \mathbf{x}^{(i)}) < 0$ will make some positive loss for incorrect prediction
- Stochastic optimization on the loss yields $\frac{\partial J^{(i)}(\boldsymbol{\theta})}{\partial \boldsymbol{\theta}} = -y^{(i)} \mathbf{x}^{(i)}$

The Perceptron

- $h(\mathbf{x}) = \text{sign}(\boldsymbol{\theta}^T \mathbf{x})$ where $\text{sign}(z) = \begin{cases} 1 & \text{if } z \geq 0 \\ -1 & \text{if } z < 0 \end{cases}$

$$h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}) = -y^{(i)}$$

- The update rule for each training example $(\mathbf{x}^{(i)}, y^{(i)})$

$$\theta_j := \theta_j - \frac{\alpha}{2} (h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}) - y^{(i)}) x_j^{(i)}$$

- For correct prediction, update will be stopped
 - For incorrect prediction update θ
- The update step can be re-written for incorrect prediction as

$$\theta_j := \theta_j + \alpha y^{(i)} x_j^{(i)}$$

- The perceptron rule: If $\mathbf{x}^{(i)}$ is misclassified, do $\boldsymbol{\theta} := \boldsymbol{\theta} + y^{(i)} \mathbf{x}^{(i)}$

Online Perceptron Algorithm

- Online learning – the learning mode where the model update is performed each time a single observation is received

Let $\boldsymbol{\theta} := [0, 0, \dots, 0]$

Repeat:

Receive training example $(\mathbf{x}^{(i)}, y^{(i)})$

If $y^{(i)}(\boldsymbol{\theta}^T \mathbf{x}^{(i)}) < 0$ //incorrect Prediction

$\boldsymbol{\theta} := \boldsymbol{\theta} + y^{(i)} \mathbf{x}^{(i)}$

Batch Learning

- The learning mode where the model update is performed after observing the entire training set

Given training data $\{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^n$

Let $\boldsymbol{\theta} := [0, 0, \dots, 0]$

Repeat:

Let $\boldsymbol{\Delta} := [0, 0, \dots, 0]$

for $i = 1, \dots, n$, do

if $y^{(i)}(\boldsymbol{\theta}^T \mathbf{x}^{(i)}) < 0$

//incorrect Prediction

• $\boldsymbol{\Delta} := \boldsymbol{\Delta} + y^{(i)} \mathbf{x}^{(i)}$

• $\boldsymbol{\Delta} := \frac{\boldsymbol{\Delta}}{n}$

//compute average update

• $\boldsymbol{\theta} := \boldsymbol{\theta} + \alpha \boldsymbol{\Delta}$

Until $\|\boldsymbol{\Delta}\|_2 < \epsilon$

Why it works

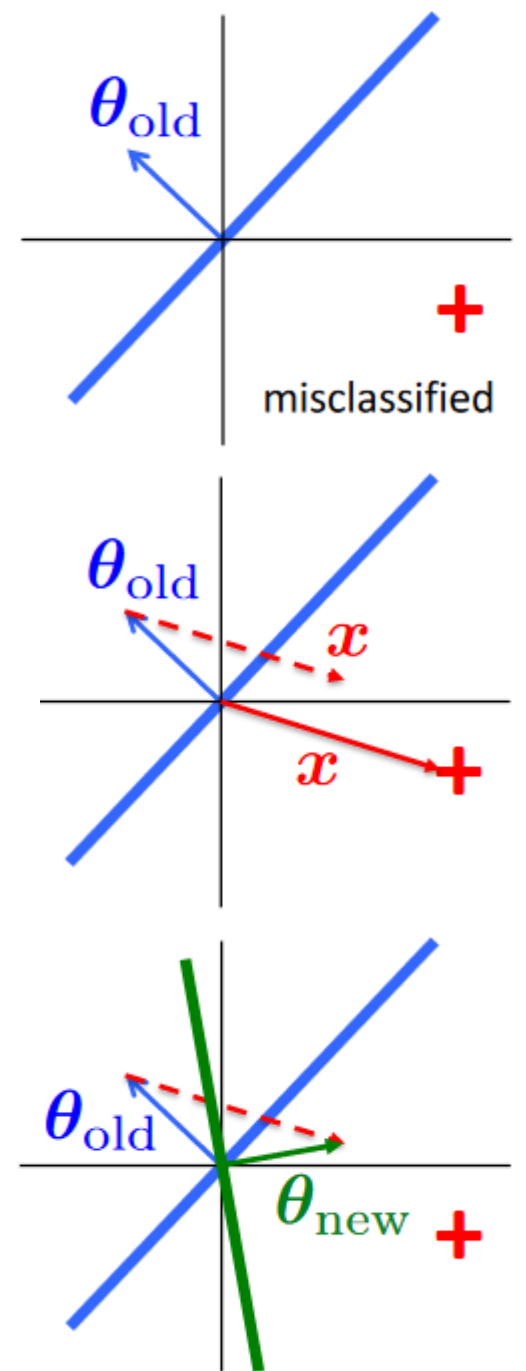
- Consider the misclassified positive example $y^{(i)} = +1$
- Perceptron wrongly predicts that $\theta_{old}^T \mathbf{x} < 0$
- Update step:

$$\theta_{new} = \theta_{old} + y\mathbf{x} = \theta_{old} + \mathbf{x} \quad (\because y = +1)$$

- Note

$$\begin{aligned}\theta_{new}^T \mathbf{x} &= (\theta_{old} + \mathbf{x})^T \mathbf{x} \\ &= \theta_{old}^T \mathbf{x} + \mathbf{x}^T \mathbf{x}\end{aligned}$$

- $\theta_{new}^T \mathbf{x}$ is less negative than $\theta_{old}^T \mathbf{x}$
- So we are progressing towards correct prediction.



Convergence of Perceptron

- **Theorem (Block & Novikoff):** If the training data is linearly separable with margin γ by a unit norm hyperplane $\boldsymbol{\theta}_*$ ($\|\boldsymbol{\theta}_*\| = 1$), then perceptron converges after $\frac{R^2}{\gamma^2}$ mistakes during training (assuming $\|\boldsymbol{x}\| \leq R$ for all $\boldsymbol{x}$)
- ⇒ • Guaranteed to find a separating hyperplane if one exists
- Convergence rate does not depend on the number of training examples or the data dimensionality.
- Depends only on the margin.

Effect of margin:- $\frac{R^2}{\gamma^2}$

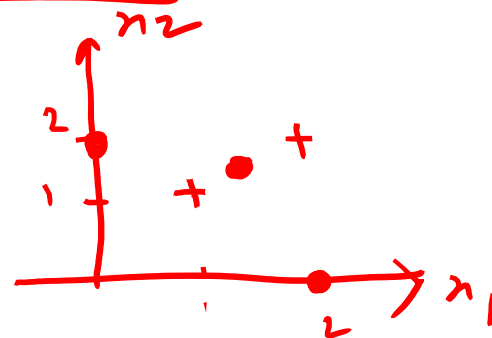
γ large $\longrightarrow$ Faster convergence

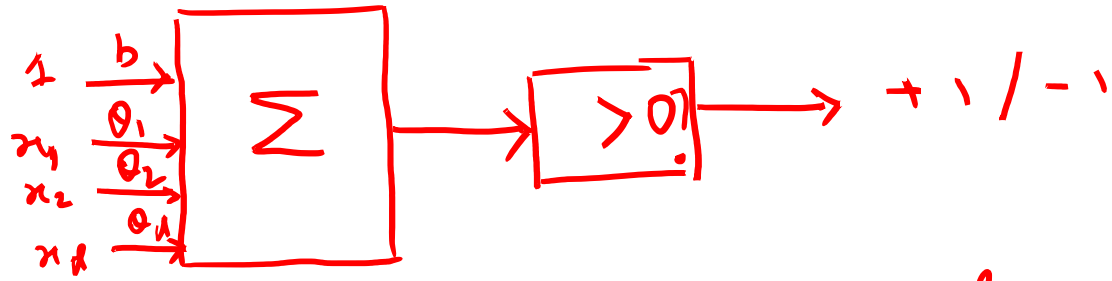
γ small $\longrightarrow$ Slower "

Effect of maximum Norm:- Large $R \longrightarrow$ Slower convergence
Small $R \longrightarrow$ Faster "

Non-linearly Separable Dataset:-

x_1	x_2	y
1	1	+1
2	2	+1
2	0	-1
0	2	-1
1.5	1.5	-1



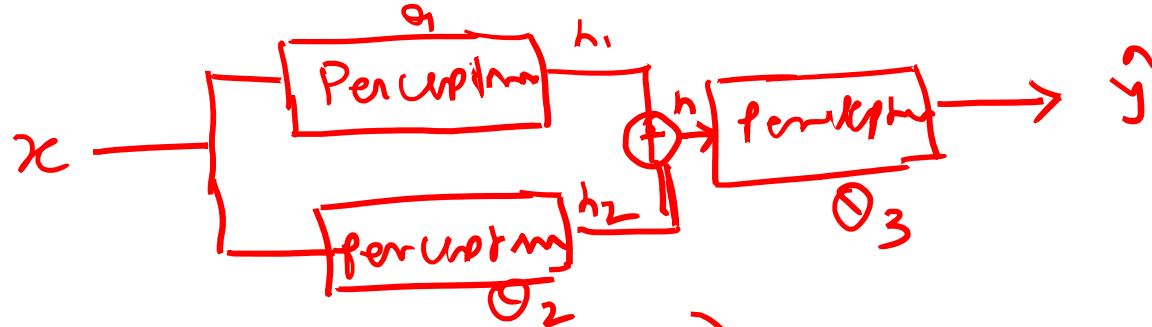


→ For linearly separable dataset - ~~the~~ perceptron will give us a classifier that ~~will~~ can classify the data perfectly.

→ For non-linearly separable dataset -
 - It makes mistake

→ update
 → Fixes for point
 → Do mistakes for other points

→ This cycle continues.

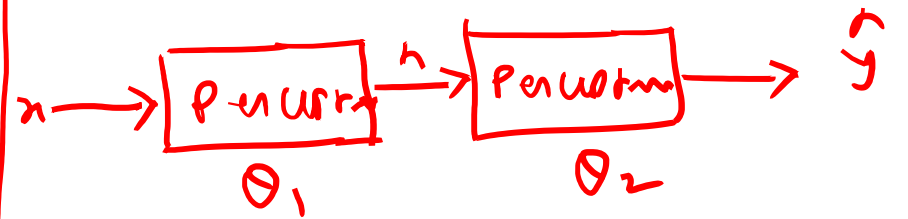


$$h_1 = \phi(\theta_1^T x + b_1)$$

$$h_2 = \phi(\theta_2^T x + b_2)$$

$$h = h_1 + h_2$$

$$\hat{y} = \phi(\theta_3^T h + b_3)$$



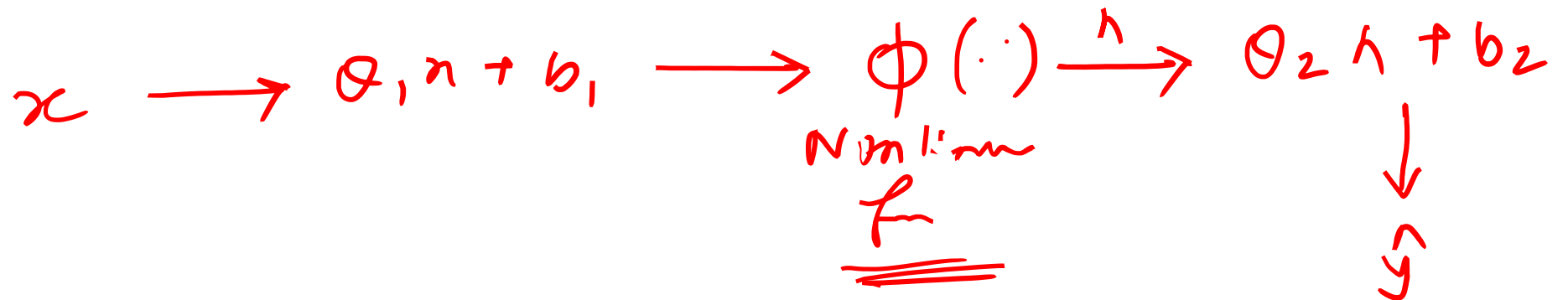
$$h = \theta_1 x + b_1$$

$$\hat{y} = \theta_2 h + b_2$$

$$\hat{y} = \theta_2(\theta_1 x + b_1) + b_2$$

$$= \underbrace{\theta_2 \theta_1}_{\theta} x + \underbrace{\theta_2 b_1 + b_2}_b$$

$$\hat{y} = \theta x + b$$



Naïve Bayes vs Perceptron

- Naïve Bayes predict classes based on the probability of the instance being that class
 - It learns $P(Y = y_k | X = \mathbf{x}_i)$
- Perceptron doesn't produce probability estimate
 - It estimates $\boldsymbol{\theta}$ from the training data
 - For new data find the sign of $\boldsymbol{\theta}^T \mathbf{x}^{(i)}$
 - Based on the sign assign a class

Logistic Regression

Logistic Regression

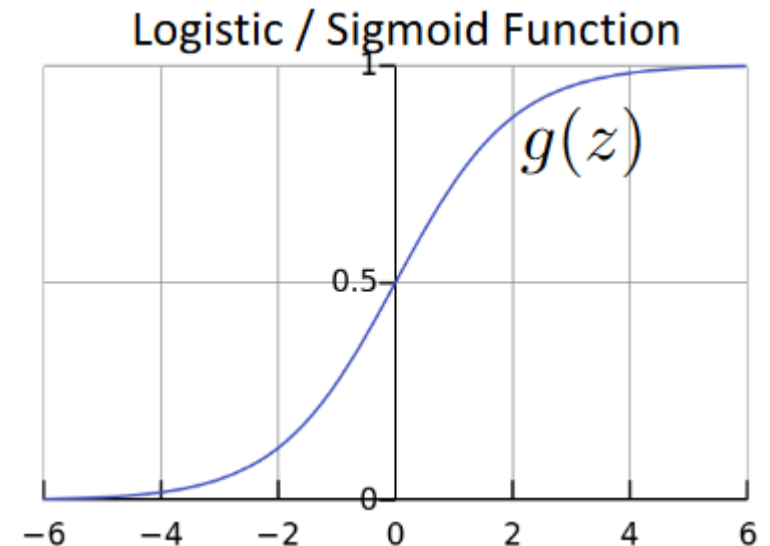
- It takes a probabilistic approach to learn a classifier (a function)
- $h_{\theta}(\mathbf{x})$ should give $p(y = 1|\mathbf{x}; \theta)$
- We want $0 \leq h_{\theta}(\mathbf{x}) \leq 1$
- Logistic regression model :

$$h_{\theta}(\mathbf{x}) = g(\theta^T \mathbf{x})$$

$$g(z) = \frac{1}{1 + e^{-z}}$$

$$h_{\theta}(\mathbf{x}) = \frac{1}{1 + e^{-\theta^T \mathbf{x}}} = \frac{e^{\theta^T \mathbf{x}}}{1 + e^{\theta^T \mathbf{x}}}$$

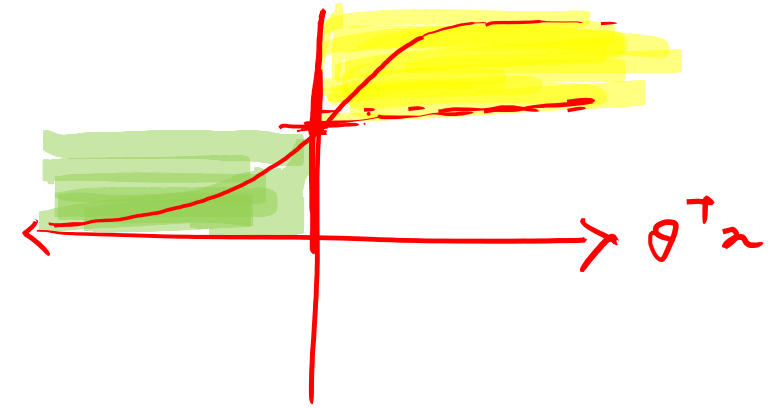
- The sigmoid first computes real-valued score and then squashes it between 0 and 1 to make it as a probability score.



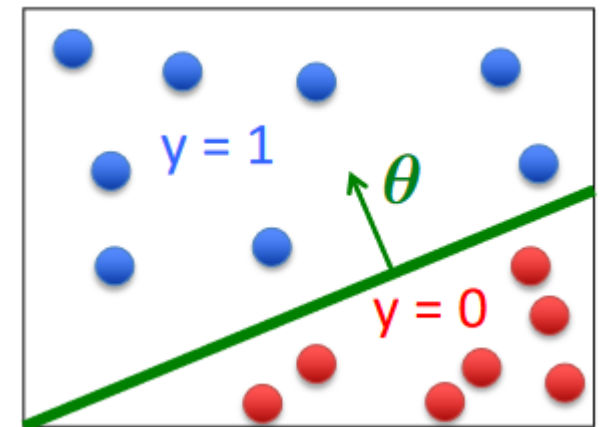
Interpreting Hypothesis Output

- $h_{\boldsymbol{\theta}}(\mathbf{x}) = \text{estimated } p(y = 1|\mathbf{x}; \boldsymbol{\theta}) = \frac{1}{1+e^{-\boldsymbol{\theta}^T \mathbf{x}}} = \frac{e^{\boldsymbol{\theta}^T \mathbf{x}}}{1+e^{\boldsymbol{\theta}^T \mathbf{x}}}$
- Note: $p(y = 0|\mathbf{x}; \boldsymbol{\theta}) + p(y = 1|\mathbf{x}; \boldsymbol{\theta}) = 1$
- So, $p(y = 0|\mathbf{x}; \boldsymbol{\theta}) = 1 - p(y = 1|\mathbf{x}; \boldsymbol{\theta}) = 1 - \frac{1}{1+e^{-\boldsymbol{\theta}^T \mathbf{x}}} = \frac{1}{1+e^{\boldsymbol{\theta}^T \mathbf{x}}}$
- The *log-odds (logits)* of the model
$$\log \frac{p(y = 1|\mathbf{x}; \boldsymbol{\theta})}{p(y = 0|\mathbf{x}; \boldsymbol{\theta})} = \log e^{\boldsymbol{\theta}^T \mathbf{x}} = \boldsymbol{\theta}^T \mathbf{x}$$
- Thus if $\boldsymbol{\theta}^T \mathbf{x} > 0$ then the positive class more probable

Logistic Regression



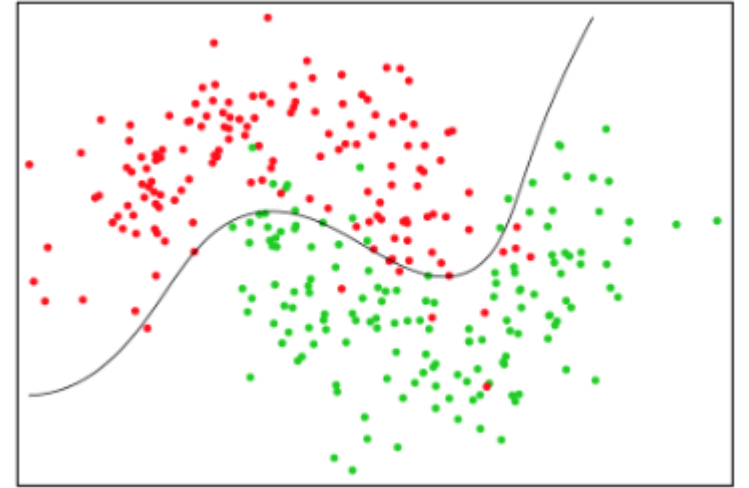
- $h_{\theta}(\mathbf{x}) = g(\theta^T \mathbf{x})$
- $g(z) = \frac{1}{1+e^{-z}}$
- $\theta^T \mathbf{x}$ should be large negative values for negative instances
- $\theta^T \mathbf{x}$ should be large positive values for positive instances
- Assume a threshold and predict
 - $y = 1$ if $h_{\theta}(\mathbf{x}) \geq 0.5$ ($\theta^T \mathbf{x} \geq 0$)
 - $y = 0$ if $h_{\theta}(\mathbf{x}) < 0.5$ ($\theta^T \mathbf{x} < 0$)



Non-linear Decision Boundary

- Can apply basis function expansion to features

$$\bullet \mathbf{x} = \begin{bmatrix} 1 \\ x_1 \\ x_2 \end{bmatrix} \rightarrow \begin{bmatrix} 1 \\ x_1 \\ x_2 \\ x_1 x_2 \\ x_1^2 \\ x_2^2 \\ x_1^2 x_2 \\ x_1 x_2^2 \\ \vdots \end{bmatrix}$$



$$h_{\theta}(x) = g(\theta^T x)$$

Let $\theta = \begin{bmatrix} -3 \\ 1 \\ 1 \end{bmatrix}$ ↓ bias



$$x_1 + x_2 < 3$$

$$y = 0$$

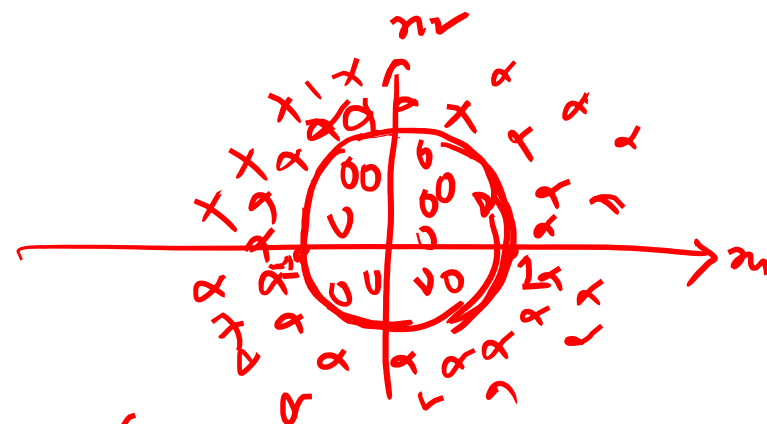
$$h_{\theta}(x) = (-3 + x_1 + x_2)$$

$$\theta^T x = 0$$

$$x_1 + x_2 = 3$$

$$\text{If } x_1 + x_2 > 3$$

$$\theta = \begin{bmatrix} -1 \\ 0 \\ 0 \\ 1 \\ 1 \end{bmatrix}$$



$$h_{\theta}(x) = g(\theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_1^2 + \theta_4 x_2^2)$$

$$\left. \begin{array}{l} \theta^T \phi(x) \geq 0 \\ x_1^2 + x_2^2 \geq 1 \end{array} \right\} y = 1$$

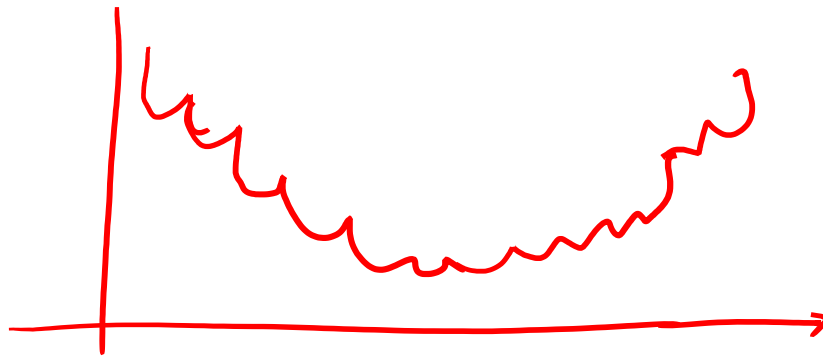
Logistic Regression Cost Function

- Should not use the squared loss as in case of linear regression

$$J(\boldsymbol{\theta}) = \frac{1}{2n} \sum_{i=1}^n (h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}) - y^{(i)})^2$$

- The logistic regression model will lead to a non-convex cost function

$$h_{\boldsymbol{\theta}}(\mathbf{x}) = \frac{1}{1 + e^{-\boldsymbol{\theta}^T \mathbf{x}}}$$



Finding the Cost Function via MLE

- Likelihood of the data is given by $L(\boldsymbol{\theta}) = \prod_{i=1}^n p(y^{(i)} | \mathbf{x}^{(i)}; \boldsymbol{\theta})$
- $\boldsymbol{\theta}$ that maximizes the likelihood

$$\boldsymbol{\theta}_{MLE} = \arg \max_{\boldsymbol{\theta}} L(\boldsymbol{\theta}) = \arg \max_{\boldsymbol{\theta}} \prod_{i=1}^n p(y^{(i)} | \mathbf{x}^{(i)}; \boldsymbol{\theta})$$

$$\begin{aligned} \boldsymbol{\theta}_{MLE} &= \arg \max_{\boldsymbol{\theta}} \log L(\boldsymbol{\theta}) = \arg \max_{\boldsymbol{\theta}} \log \prod_{i=1}^n p(y^{(i)} | \mathbf{x}^{(i)}; \boldsymbol{\theta}) \\ &= \arg \max_{\boldsymbol{\theta}} \sum_{i=1}^n \log p(y^{(i)} | \mathbf{x}^{(i)}; \boldsymbol{\theta}) \end{aligned}$$

Finding the Cost Function via MLE

- Each label $y^{(i)}$ is binary with probability $h_{\theta}(\mathbf{x}^{(i)})$
- Assume Bernoulli likelihood

$$\begin{aligned} p(\mathbf{y}|\mathbf{X}, \boldsymbol{\theta}) &= \prod_{i=1}^n p(y^{(i)} | \mathbf{x}^{(i)}; \boldsymbol{\theta}) \\ &= \prod_{i=1}^n h_{\theta}(\mathbf{x}^{(i)})^{y^{(i)}} \left(1 - h_{\theta}(\mathbf{x}^{(i)})\right)^{1-y^{(i)}} \end{aligned}$$

- The log-likelihood

$$l(\boldsymbol{\theta}) = \sum_{i=1}^n y^{(i)} \log h_{\theta}(\mathbf{x}^{(i)}) + (1 - y^{(i)}) \log (1 - h_{\theta}(\mathbf{x}^{(i)}))$$

The Cost Function

- Maximizing $l(\boldsymbol{\theta})$ is equivalent to minimizing the NLL

$$J(\boldsymbol{\theta}) = NLL(\boldsymbol{\theta}) = - \sum_{i=1}^n y^{(i)} \log h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}) + (1 - y^{(i)}) \log (1 - h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}))$$

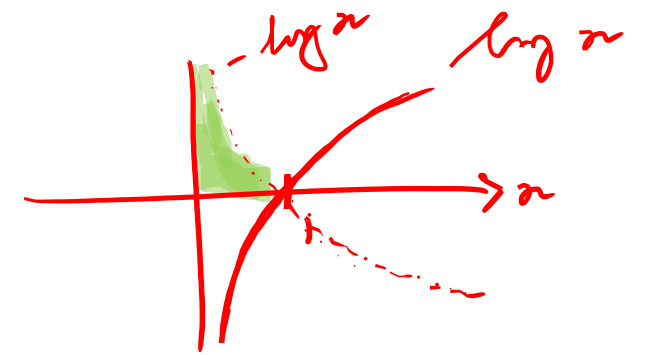
- Cost of a single instance

$$cost(h_{\boldsymbol{\theta}}(\mathbf{x}), y) = \begin{cases} -\log h_{\boldsymbol{\theta}}(\mathbf{x}) & \text{if } y = 1 \\ -\log (1 - h_{\boldsymbol{\theta}}(\mathbf{x})) & \text{if } y = 0 \end{cases}$$

- The objective function

$$J(\boldsymbol{\theta}) = \sum_{i=1}^n cost(h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}), y^{(i)})$$

Intuition



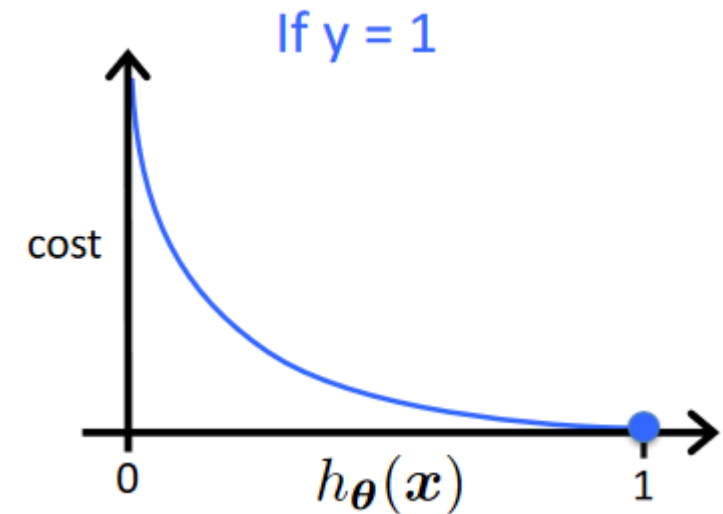
$$\bullet \text{ cost}(h_{\theta}(\mathbf{x}), y) = \begin{cases} -\log h_{\theta}(\mathbf{x}) & \text{if } y = 1 \\ -\log(1 - h_{\theta}(\mathbf{x})) & \text{if } y = 0 \end{cases}$$

if $y = 1$

if $y = 0$

- If $y = 1$

- $\text{cost} = 0$ for correct prediction
- As $h_{\theta}(\mathbf{x}) \rightarrow 0$, $\text{cost} \rightarrow \infty$
- Mistakes should get large penalties
- e.g., predict $h_{\theta}(\mathbf{x}) = 0$, but $y = 1$

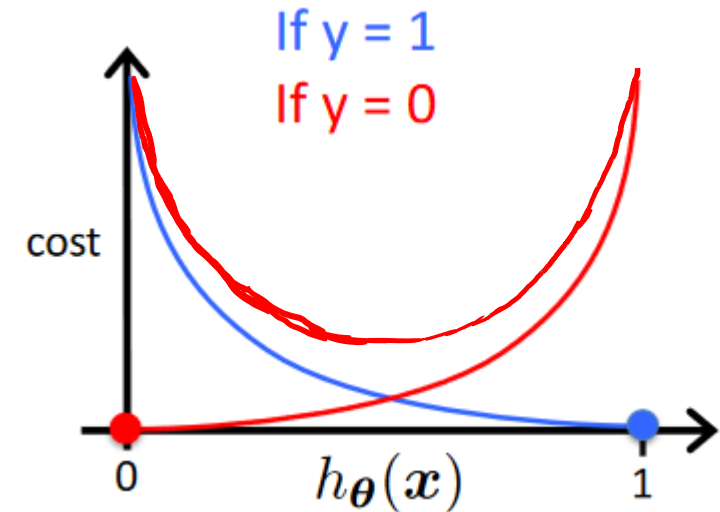


Intuition

- $cost(h_{\theta}(x), y) = \begin{cases} -\log h_{\theta}(x) \\ -\log (1 - h_{\theta}(x)) \end{cases}$

if $y = 1$
if $y = 0$

- If $y = 0$
 - $cost = 0$ for correct prediction
 - As $(1 - h_{\theta}(x)) \rightarrow 0$, $cost \rightarrow \infty$
 - Mistakes should get large penalties
 - e.g., predict $h_{\theta}(x) = 0$, but $y = 1$



Logistic Regression Cost fn. $z^{(i)} = \theta^T x^{(i)}$; $\sigma(z) = \frac{1}{1 + e^{-z}}$

$$J(\theta) = - \sum_{i=1}^n \left[y^{(i)} \log \sigma(z^{(i)}) + (1 - y^{(i)}) \log (1 - \sigma(z^{(i)})) \right]$$

$$= - \sum_{i=1}^n \left[\underbrace{y^{(i)} \log \frac{1}{1 + e^{-z^{(i)}}}}_{\text{1st term}} + \underbrace{(1 - y^{(i)}) \log \frac{e^{-z^{(i)}}}{1 + e^{-z^{(i)}}}}_{\text{2nd term}} \right]$$

1st Term:- $y^{(i)} \log \frac{1}{1 + e^{-z^{(i)}}} = - y^{(i)} \log (1 + e^{-z^{(i)}})$

2nd term:- $\log \frac{e^{-z}}{1 + e^{-z}} = \log e^{-z} - \log (1 + e^{-z})$

$$= -z - \log (1 + e^{-z})$$

$$(1 - y^{(i)}) \log \frac{e^{-z}}{1 + e^{-z}} = - (1 - y^{(i)}) z^{(i)} - (1 - y^{(i)}) \log (1 + e^{-z^{(i)}})$$

$$J(\theta) = \underbrace{- y^{(i)} \log (1 + e^{-z^{(i)}})}_{\text{1st term}} - \underbrace{(1 - y^{(i)}) z^{(i)} - (1 - y^{(i)}) \log (1 + e^{-z^{(i)}})}_{\text{2nd term}}$$

$$= - \sum_{i=1}^n \left[-\log(1 + e^{-z^{(i)}}) - (1 - y^{(i)}) z^{(i)} \right]$$

$$= \sum_{i=1}^n \left[\log(1 + e^{-z^{(i)}}) + (1 - y^{(i)}) z^{(i)} \right]$$

$$= \sum_{i=1}^n \left[\log(1 + e^{z^{(i)}}) - \log e^{z^{(i)}} + z^{(i)} - y^{(i)} z^{(i)} \right] = \frac{1 + e^z}{e^z}$$

$$J = \sum_{i=1}^n \left[\log(1 + e^{z^{(i)}}) - y^{(i)} z^{(i)} \right]$$

$$\frac{\partial J}{\partial \theta} = \frac{\partial}{\partial \theta} \left[\sum_{i=1}^n \left(\log(1 + e^{\theta^T x^{(i)}}) - y^{(i)} \theta^T x^{(i)} \right) \right]$$

$$= \sum_{i=1}^n \left(\frac{e^{\theta^T x^{(i)}}}{1 + e^{\theta^T x^{(i)}}} \cdot x^{(i)} - y^{(i)} x^{(i)} \right)$$

$$\frac{\partial J}{\partial \theta} = \sum_{i=1}^n \left(\lambda_{\theta}(x^{(i)}) - y^{(i)} \right) x^{(i)}$$

$$= X^T \left(\lambda_{\theta}(\vec{x}^{(i)}) - \vec{y} \right) \longrightarrow 0$$

we cannot get a closed form solution for $\vec{\theta}$

$$\text{by } \frac{\partial J}{\partial \theta} = 0.$$

Logistic Regression (contd.)

MAP formulation

— use a prior on $\vec{\theta}$

$$p(\vec{\theta}) = \mathcal{N}(\vec{0}, \lambda^{-1} \bar{I}_d) \propto \exp(-\lambda \vec{\theta}^T \vec{\theta})$$

map objective: MLE objective + $\log p(\theta)$

$$\begin{aligned} J_{\text{map}}(\theta) &= J_{\text{MLE}}(\theta) + \log p(\theta) \\ &= \sum_{i=1}^n \left[\log(1 + e^{z^{(i)}}) - y^{(i)} z^{(i)} \right] - \lambda \vec{\theta}^T \vec{\theta} \end{aligned}$$

$$J_{\text{map}}(\theta) = - \left[\sum_{i=1}^n (y^{(i)} \theta^T x^{(i)} - \log(1 + \exp(\theta^T x^{(i)}))) + \lambda \vec{\theta}^T \vec{\theta} \right]$$

Regularized Logistic Regression

- $J(\boldsymbol{\theta}) = -\sum_{i=1}^n y^{(i)} \log h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}) + (1 - y^{(i)}) \log (1 - h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}))$
- We can regularize the logistic regression as

$$\begin{aligned} J_{reg}(\boldsymbol{\theta}) &= J(\boldsymbol{\theta}) + \lambda \sum_{j=1}^d \theta_j^2 \\ &= J(\boldsymbol{\theta}) + \lambda \|\boldsymbol{\theta}\|_2^2 \end{aligned}$$

$$\begin{aligned} J_{reg}(\boldsymbol{\theta}) &= -\sum_{i=1}^n y^{(i)} \log h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}) + (1 - y^{(i)}) \log (1 - h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)})) + \lambda \sum_{j=1}^d \theta_j^2 \end{aligned}$$

Gradient Descent for Logistic Regression

- Objective is to find $\min_{\theta} J(\theta)$

- Initialize θ
- Repeat until convergence

$$\theta_j \leftarrow \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta)$$

simultaneous update
for $j = 0 \dots d$

$$\frac{\partial}{\partial \theta_j} J(\theta) = - \sum_{i=1}^n (y^{(i)} - h_{\theta}(x^{(i)})) x^{(i)}$$

Gradient Descent for Logistic Regression

- Objective is to find $\min_{\theta} J(\theta)$

$$\theta_0 \leftarrow \theta_0 - \alpha \sum_{i=1}^n \left(h_{\theta} \left(\mathbf{x}^{(i)} \right) - y^{(i)} \right)$$

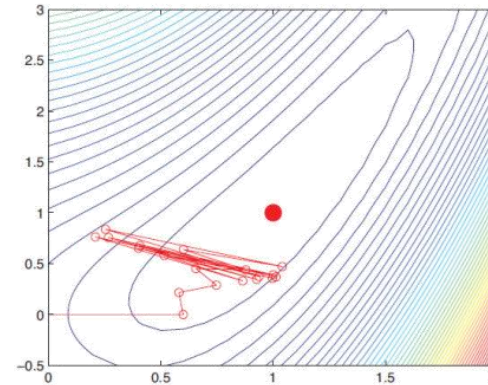
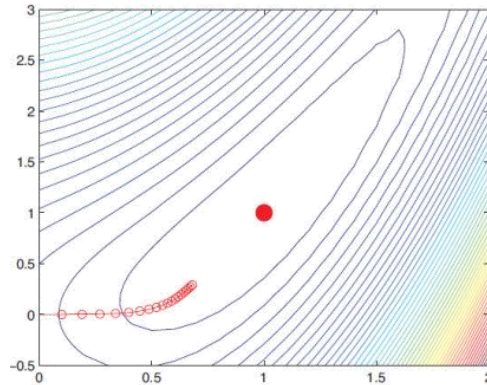
← unregularized

$$\theta_j \leftarrow \theta_j - \alpha \left[\sum_{i=1}^n \left(h_{\theta} \left(\mathbf{x}^{(i)} \right) - y^{(i)} \right) x_j^{(i)} - \frac{\lambda}{n} \theta_j \right]$$

← Regularized

More on Gradient Descent

- GD can converge slowly and is also sensitive to the step size



- Choose optimal step size by line-search technique
- Add a momentum term to the updates
- Use second order methods (e.g., Newton's method) to utilize the curvature of the loss function

Gradient Descent with Momentum



$$v_0 = 0$$

$$v_t = -\eta \sum_{k=0}^{t-1} \beta^k \nabla J(\theta_{t-1-k})$$

Exponentially weighted moving average of past gradients.

$$\text{velocity: } v_{t+1} := \beta v_t - \eta \nabla J(\theta_t)$$

$$\theta_{t+1} := \theta_t + v_{t+1}$$

$\beta \in (0, 1)$: damping coefficient or Friction (Typically '0.9')

v : velocity stores exponentially decaying past gradients

η : Time-step scaling / learning rate

Gradient Descent with Momentum

① if gradients point roughly in the same direction
— we double reinforces each other
— speed increases

② If gradient oscillates — +ve and -ve directions cancel
— oscillation is damped.

$$v_{t+1} = \beta v_t - \eta \nabla J(\theta_t)$$

$$\theta_{t+1} = \theta_t + v_{t+1}$$

$\beta = 0 \rightarrow$ No momentum $\rightarrow$ No inertia

$\beta \approx 1 \rightarrow$ No friction / very little fric.
Oscillation explodes $\rightarrow$ instability.

$\beta = 0.9 \rightarrow 90\% \rightarrow$ stable convergence